

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Đề tài

THIẾT KẾ CẦU CHUYỂN ĐỔI GIỮA HAI GIAO THỨC

AHB VÀ APB

GVHD : ThS. LÊ MINH THÀNH

Sinh viên : LÊ HỮU LƯƠNG

MSSV: 22119195

NGUYỄN VĂN QUÂN

MSSV: 22119220

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Đề tài

THIẾT KẾ CẦU CHUYỂN ĐỔI GIỮA HAI GIAO THỨC

AHB VÀ APB

GVHD : ThS. LÊ MINH THÀNH

Sinh viên : LÊ HỮU LƯƠNG

MSSV: 22119195

NGUYỄN VĂN QUÂN

MSSV: 22119220

TP. HỒ CHÍ MINH – 06/2026



TP. HCM, ngày 12 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Lê Hữu Lương

MSSV: 22119195

Nguyễn Văn Quân

MSSV: 22119220

Ngành: Công nghệ kỹ thuật máy tính

Lớp: 221191A

Giảng viên hướng dẫn: ThS. Lê Minh Thành

Ngày nhận đề tài: 14/03/2026

Ngày nộp đề tài: 12/06/2026

1. Tên đề tài: Thiết kế cầu chuyển đổi giữa hai giao thức AHB và APB

2. Các số liệu, tài liệu ban đầu

- Kiến thức cơ bản về các môn Thiết kế hệ thống và vi mạch tích hợp, Thiết kế mạch tích hợp VLSI, Kỹ thuật số,...

3. Nội dung thực hiện đề tài

- Tìm hiểu tổng quan về kiến trúc bus AMBA, giao thức AHB và giao thức APB.
- Phân tích nguyên lý hoạt động và cơ chế truyền dữ liệu giữa hai giao thức AHB và APB.
- Thiết kế cầu chuyển đổi giữa hai giao thức AHB và APB.
- Xây dựng môi trường kiểm thử UVM để kiểm tra chức năng của cầu chuyển đổi AHB – APB trên công cụ QuestaSim.
- Tổng hợp thiết kế trên Vivado để đánh giá tài nguyên sử dụng, tần số hoạt động, độ trễ và công suất tiêu thụ.
- Hoàn thiện báo cáo đồ án tốt nghiệp.

GIẢNG VIÊN HƯỚNG DẪN

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

Đề tài: Thiết kế cầu chuyển đổi giữa hai giao thức AHB và APB.

Sinh viên: + Lê Hữu Lương

MSSV: 22119195

+ Nguyễn Văn Quân

MSSV: 22119220

Hướng dẫn: ThS. Lê Minh Thành

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Sinh viên có học tập trung, nghiêm túc, đạt được kết quả tốt.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu và được phép bảo vệ.

Tp. HCM, ngày 22 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Lê Minh Thành

Lê Minh Thành

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Lê Minh Thành, người đã trực tiếp hướng dẫn, định hướng và hỗ trợ chúng em trong quá trình thực hiện đề tài. Những góp ý của thầy đã giúp chúng em khắc phục các khó khăn về kỹ thuật, đồng thời hoàn thiện nội dung báo cáo rõ ràng và hợp lý hơn.

Chúng em xin chân thành cảm ơn quý thầy cô Khoa Điện – Điện tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh đã truyền đạt những kiến thức nền tảng trong suốt quá trình học tập. Đây là cơ sở quan trọng giúp chúng em có đủ kiến thức để tiếp cận và triển khai đề tài.

Chúng em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên và tạo điều kiện thuận lợi để chúng em hoàn thành đồ án trong thời gian qua.

Do thời gian thực hiện và kinh nghiệm thực tế còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý của quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn.

Nhóm thực hiện đề tài

Lê Hữu Lương

Nguyễn Văn Quân

TÓM TẮT ĐỀ TÀI

Trong hệ thống SoC, các khối chức năng thường có tốc độ và yêu cầu xử lý khác nhau. Các khối hiệu năng cao cần truyền dữ liệu nhanh, trong khi các ngoại vi thường hoạt động chậm hơn và có cơ chế giao tiếp đơn giản hơn. Vì vậy, hệ thống cần một khối trung gian để kết nối và chuyển đổi giao tiếp giữa hai nhóm thành phần này.

Đề tài tập trung thiết kế cầu chuyển đổi AHB – APB. Trong đó, AHB được dùng cho phía bus tốc độ cao, còn APB được dùng để kết nối các ngoại vi có yêu cầu truyền dữ liệu thấp hơn. Cầu chuyển đổi có nhiệm vụ tiếp nhận giao dịch từ AHB Master, tạo chu kỳ truy cập tương ứng trên APB và trả kết quả phản hồi về phía AHB. Thiết kế cũng xử lý sự khác biệt giữa cơ chế đường ống của AHB và chu kỳ truyền đơn giản của APB.

Để kiểm tra chức năng, thiết kế được xác minh bằng môi trường UVM với các kịch bản gồm giao dịch đơn, chuỗi giao dịch liên tiếp và trường hợp phản hồi lỗi. Sau đó, thiết kế được tổng hợp trên FPGA để đánh giá tài nguyên sử dụng, tần số hoạt động và công suất tiêu thụ.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT ĐỀ TÀI	ii
MỤC LỤC.....	iii
DANH MỤC CÁC TỪ VIẾT TẮT.....	v
DANH MỤC CÁC HÌNH ẢNH	vi
DANH MỤC CÁC BẢNG BIỂU	viii
CHƯƠNG 1: TỔNG QUAN.....	1
1.1 GIỚI THIỆU	1
1.2 MỤC TIÊU	1
1.3 GIỚI HẠN	2
1.4 BỐ CỤC ĐỒ ÁN	2
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	3
2.1 VAI TRÒ CỦA KIẾN TRÚC BUS TRONG HỆ THỐNG TÍCH HỢP	3
2.2 CÁC ĐẶC ĐIỂM CỦA GIAO THỨC AHB.....	4
2.2.1 Cấu trúc pha địa chỉ và pha dữ liệu	4
2.2.2 Trạng thái giao dịch và truyền burst.....	6
2.2.3 Phản hồi giao dịch trên AHB.....	8
2.3 CÁC ĐẶC ĐIỂM CỦA GIAO THỨC APB.....	8
2.3.1 Chu kỳ SETUP và ACCESS	8
2.3.2 Cơ chế chờ bằng PREADY	10
2.3.3 Phản hồi lỗi bằng PSLVERR.....	11
2.4 CƠ SỞ LÝ THUYẾT VỀ MÔI TRƯỜNG UVM	11
2.4.1 Mô hình kiểm thử theo giao dịch	11
2.4.2 Vai trò của các thành phần trong môi trường UVM.....	12
2.4.3 Cơ chế pha trong UVM	13
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	15
3.1 YÊU CẦU CỦA HỆ THỐNG	15
3.2 SƠ ĐỒ KHỐI HỆ THỐNG	15
3.3 THIẾT KẾ CHI TIẾT KHỐI	19

3.3.1 Thiết kế khối giao diện tớ AHB	19
3.3.2 Thiết kế máy trạng thái APB	27
3.4 THIẾT KẾ MÔI TRƯỜNG UVM.....	31
3.4.1 Tổng quan kiến trúc môi trường UVM	31
3.4.2 Thiết kế AHB Agent	33
3.4.3 Thiết kế APB Agent.....	37
3.4.4 Thiết kế Scoreboard.....	41
CHƯƠNG 4: KẾT QUẢ	43
4.1 KẾT QUẢ MÔ PHỎNG CHỨC NĂNG.....	43
4.1.1 Các trường hợp kiểm thử chức năng của hệ thống.....	43
4.1.2 Kết quả mô phỏng	47
4.2 ĐÁNH GIÁ CẦU CHUYỂN ĐỔI AHB – APB TRÊN FPGA.....	58
4.2.1 Đánh giá tài nguyên sử dụng.....	59
4.2.2 Đánh giá tần số hoạt động và độ trễ	60
4.2.3 Đánh giá công suất tiêu thụ	61
4.2.4 Tổng hợp kết quả triển khai.....	62
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	63
5.1 KẾT LUẬN.....	63
5.2 HƯỚNG PHÁT TRIỂN.....	63
TÀI LIỆU THAM KHẢO.....	64
PHỤ LỤC.....	65

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
AMBA	Advanced Microcontroller Bus Architecture
ARM	Advanced RISC Machine
AXI	Advanced eXtensible Interface
SoC	System on Chip
IP	Intellectual Property
FSM	Finite State Machine
UART	Universal Asynchronous Receiver/Transmitter
SPI	Serial Peripheral Interface
I2C	Inter-Integrated Circuit
GPIO	General-Purpose Input/Output
UVM	Universal Verification Methodology
DUT	Design Under Test
LUT	Look-Up Table
FF	Flip-Flop
IO	Input Output
BUFG	Global Clock Buffer
WNS	Worst Negative Slack
WHS	Worst Hold Slack
WPWS	Worst Pulse Width Slack
TNS	Total Negative Slack

DANH MỤC CÁC HÌNH ẢNH

Hình 2.1:	Giản đồ dạng sóng của giao dịch đọc AHB	5
Hình 2.2:	Giản đồ dạng sóng của giao dịch ghi AHB	5
Hình 2.3:	Giản đồ dạng sóng các trạng thái giao dịch AHB	7
Hình 2.4:	Sơ đồ trạng thái của giao thức APB	9
Hình 2.5:	Giản đồ dạng sóng của giao dịch ghi APB	9
Hình 2.6:	Giản đồ dạng sóng của giao dịch đọc APB	10
Hình 2.7:	Kiến trúc tổng quát của môi trường UVM.....	12
Hình 2.8:	Trình tự thực thi các pha trong UVM	13
Hình 3.1:	Sơ đồ khối tổng quát cầu chuyển đổi AHB – APB	15
Hình 3.2:	Các tín hiệu đầu vào và đầu ra của cầu chuyển đổi AHB – APB	16
Hình 3.3:	Kiến trúc nội bộ của khối giao diện tứ AHB.....	19
Hình 3.4:	Các tín hiệu đầu vào và đầu ra của khối giao diện tứ AHB	19
Hình 3.5:	Sơ đồ logic khối phát hiện giao dịch hợp lệ.....	20
Hình 3.6:	Giản đồ dạng sóng khối phát hiện giao dịch hợp lệ.....	20
Hình 3.7:	Sơ đồ logic khối đường ống.....	21
Hình 3.8:	Giản đồ dạng sóng khối đường ống	22
Hình 3.9:	Sơ đồ logic của khối giải mã địa chỉ.....	23
Hình 3.10:	Giản đồ dạng sóng của khối giải mã địa chỉ	24
Hình 3.11:	Sơ đồ logic khối phản hồi tín hiệu	25
Hình 3.12:	Giản đồ dạng sóng của phản hồi tín hiệu	26
Hình 3.13:	Các tín hiệu đầu vào và đầu ra của máy trạng thái APB.....	27
Hình 3.14:	Sơ đồ máy trạng thái APB.....	28
Hình 3.15:	Sơ đồ môi trường UVM của cầu chuyển đổi AHB – APB	32
Hình 3.16:	Kiến trúc nội bộ AHB Agent.....	33

Hình 3.17:	Lưu đồ chương trình con thực thi giao dịch của AHB Driver	34
Hình 3.18:	Lưu đồ hoạt động của AHB Driver.....	35
Hình 3.19:	Lưu đồ chương trình con theo dõi và thu thập giao dịch	36
Hình 3.20:	Lưu đồ hoạt động của AHB Monitor	37
Hình 3.21:	Kiến trúc nội bộ APB Agent	37
Hình 3.22:	Lưu đồ chương trình con phản hồi giao dịch.....	38
Hình 3.23:	Lưu đồ hoạt động của APB Driver.....	39
Hình 3.24:	Lưu đồ chương trình con theo dõi và thu thập giao dịch	40
Hình 3.25:	Lưu đồ hoạt động của APB Monitor	41
Hình 4.1:	Mô phỏng cầu chuyển đổi AHB – APB ở kịch bản giao dịch ghi đơn	47
Hình 4.2:	Mô phỏng cầu chuyển đổi AHB – APB ở kịch bản giao dịch đọc đơn ...	48
Hình 4.3:	Kết quả kiểm tra của Scoreboard ở testcase 1	50
Hình 4.4:	Mô phỏng cầu AHB – APB ở kịch bản chuỗi giao dịch ghi liên tiếp	51
Hình 4.5:	Kết quả kiểm tra của Scoreboard ở testcase 2	53
Hình 4.6:	Mô phỏng cầu AHB – APB ở kịch bản chuỗi giao dịch đọc liên tiếp	54
Hình 4.7:	Kết quả kiểm tra của Scoreboard ở testcase 3	55
Hình 4.8:	Mô phỏng cầu AHB – APB ở kịch bản phản hồi báo lỗi từ APB Slave ..	56
Hình 4.9:	Kết quả kiểm tra của Scoreboard ở testcase 4	58
Hình 4.10:	Cấu hình ràng buộc định thời cho cầu chuyển đổi AHB – APB.....	59
Hình 4.11:	Kết quả sử dụng tài nguyên sau triển khai thiết kế	59
Hình 4.12:	Kết quả tóm tắt phân tích định thời của thiết kế	60
Hình 4.13:	Kết quả báo cáo công suất tiêu thụ của thiết kế.....	61

DANH MỤC CÁC BẢNG BIỂU

Bảng 2.1: Các trạng thái giao dịch của AHB	6
Bảng 2.2: Các kiểu truyền burst của AHB	7
Bảng 3.1: Tín hiệu đầu vào và đầu ra của hệ thống	17
Bảng 3.2: Tín hiệu nội bộ giữa khối giao diện tới AHB và máy trạng thái APB	18
Bảng 3.3: Phân bổ không gian địa chỉ và các tín hiệu giải mã chọn ngoại vi APB	25
Bảng 3.4: Các trạng thái trong máy trạng thái APB	28
Bảng 3.5: Các điều kiện kiểm thử chức năng của Scoreboard	42
Bảng 4.1: Tóm tắt các kịch bản kiểm thử chức năng hệ thống	43
Bảng 4.2: Các kịch bản kiểm thử cho cầu chuyển đổi AHB – APB	44
Bảng 4.3: Tổng hợp kết quả triển khai cầu chuyển đổi AHB – APB trên FPGA	62

Chương 1

TỔNG QUAN

1.1 GIỚI THIỆU

Các thiết kế SoC hiện nay thường tích hợp nhiều thành phần xử lý và ngoại vi trên cùng một vi mạch. Mỗi nhóm khối chức năng lại có yêu cầu giao tiếp khác nhau. Các khối như bộ xử lý, bộ nhớ và DMA cần trao đổi dữ liệu với tốc độ cao, trong khi các ngoại vi như UART, GPIO, Timer hoặc SPI thường chỉ thực hiện các tác vụ điều khiển đơn giản hơn. Vì vậy, việc sử dụng một kiến trúc bus phù hợp là cần thiết để tổ chức quá trình truyền dữ liệu giữa các khối một cách ổn định và dễ mở rộng.

AMBA là kiến trúc bus được sử dụng phổ biến trong các thiết kế SoC nhằm chuẩn hóa giao tiếp giữa các thành phần phần cứng. Trong họ giao thức này, AHB thường được dùng cho các khối yêu cầu băng thông cao, còn APB được dùng cho các ngoại vi có tốc độ thấp và cấu trúc điều khiển đơn giản hơn. Tuy nhiên, do AHB và APB có cơ chế hoạt động khác nhau, hai bus này cần một khối trung gian để chuyển đổi giao dịch. Cầu chuyển đổi AHB – APB đảm nhiệm việc nhận yêu cầu từ AHB Master, tạo chu kỳ truy cập tương ứng trên APB và trả kết quả phản hồi về phía AHB.

Trong quá trình thiết kế phần cứng số, xác minh chức năng là bước quan trọng để đảm bảo mạch hoạt động đúng trước khi triển khai. Với các thiết kế nhỏ, testbench Verilog có thể được sử dụng để kiểm tra một số trường hợp cơ bản. Tuy nhiên, khi số lượng tình huống kiểm thử tăng lên, cách kiểm tra thủ công sẽ khó quản lý, khó mở rộng và dễ bỏ sót lỗi.

Để giải quyết vấn đề này, đề tài sử dụng UVM để xây dựng môi trường xác minh cho cầu chuyển đổi AHB – APB. UVM cho phép mô tả kiểm thử ở mức giao dịch, tách biệt giữa phần tạo kích thích, điều khiển tín hiệu, quan sát dữ liệu và đối chiếu kết quả. Nhờ đó, các kịch bản như đọc, ghi, truyền liên tiếp và phản hồi lỗi có thể được tổ chức rõ ràng hơn.

1.2 MỤC TIÊU

Mục tiêu của đề tài là thiết kế cầu chuyển đổi AHB – APB có khả năng chuyển tiếp giao dịch đúng giữa bus AHB và bus APB. Bên cạnh đó, đề tài xây dựng môi trường xác minh UVM trên QuestaSim để kiểm tra chức năng thiết kế thông qua các

kịch bản kiểm thử có tổ chức. Sau quá trình mô phỏng, thiết kế được tổng hợp bằng Vivado trên nền tảng FPGA nhằm đánh giá các chỉ tiêu phần cứng như tài nguyên sử dụng, khả năng hoạt động ở tần số yêu cầu và công suất tiêu thụ.

1.3 GIỚI HẠN

Phạm vi thực hiện của đề tài được xác định theo một số giới hạn nhất định. Thiết kế chỉ làm việc với một AHB Master nên không triển khai cơ chế phân quyền truy cập cho nhiều Master. Ở phía APB, hệ thống hỗ trợ bốn Slave và việc lựa chọn Slave được thực hiện dựa trên vùng địa chỉ tương ứng. Hai giao thức AHB và APB được giả định hoạt động cùng một xung nhịp, vì vậy thiết kế chưa xử lý trường hợp đồng bộ giữa hai miền xung nhịp riêng biệt. Phần xác minh chức năng được thực hiện bằng môi trường UVM trên QuestaSim thông qua các kịch bản kiểm thử đã xây dựng. Sau cùng, thiết kế được tổng hợp trên Vivado để đánh giá tài nguyên sử dụng trên FPGA.

1.4 BỐ CỤC ĐỒ ÁN

Chương 1: Tổng quan: Trình bày vai trò của kiến trúc bus và lý do thực hiện đề tài cầu chuyển đổi AHB – APB. Chương này cũng nêu mục tiêu, phạm vi và bố cục của đề tài.

Chương 2: Cơ sở lý thuyết: Trình bày các kiến thức nền tảng liên quan đến đề tài, gồm kiến trúc AMBA, nguyên lý hoạt động của giao thức AHB, APB và phương pháp xác minh UVM.

Chương 3: Thiết kế hệ thống: Mô tả yêu cầu thiết kế, sơ đồ khối tổng thể, cấu trúc chi tiết của cầu chuyển đổi AHB – APB và kiến trúc môi trường xác minh UVM.

Chương 4: Kết quả: Trình bày kết quả mô phỏng chức năng, các kịch bản kiểm thử trong môi trường UVM và kết quả đánh giá thiết kế trên FPGA.

Chương 5: Kết luận và hướng phát triển: Tổng kết các kết quả đã đạt được, nêu những hạn chế còn tồn tại và đề xuất hướng phát triển cho đề tài trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 VAI TRÒ CỦA KIẾN TRÚC BUS TRONG HỆ THỐNG TÍCH HỢP

Các thiết kế vi mạch hiện nay thường tích hợp nhiều khối chức năng trên cùng một chip, bao gồm khối xử lý, bộ nhớ, bộ điều khiển truy cập dữ liệu và các ngoại vi. Những khối này không có cùng yêu cầu về tốc độ truyền, độ phức tạp điều khiển và mức tiêu thụ tài nguyên. Vì vậy, việc tổ chức giao tiếp giữa các thành phần cần được thực hiện thông qua một kiến trúc bus phù hợp.

Kiến trúc bus đóng vai trò là đường truyền chung để các khối chức năng trao đổi dữ liệu và tín hiệu điều khiển. Nếu không có một chuẩn giao tiếp thống nhất, mỗi khối có thể phải sử dụng một kiểu kết nối riêng, làm cho thiết kế khó mở rộng và khó kiểm tra. Việc sử dụng bus chuẩn giúp giảm độ phức tạp khi tích hợp nhiều thành phần, đồng thời hỗ trợ việc tái sử dụng các khối thiết kế trong những hệ thống khác nhau.

Trong đề tài này, kiến trúc AMBA được sử dụng làm cơ sở cho việc thiết kế cầu chuyển đổi giữa hai giao thức AHB và APB [3]. AHB phù hợp với phía giao tiếp tốc độ cao, nơi cần truyền dữ liệu nhanh giữa Master và hệ thống. Ngược lại, APB được dùng cho các ngoại vi có yêu cầu truyền dữ liệu thấp hơn và cấu trúc điều khiển đơn giản hơn.

Sự khác biệt giữa AHB và APB tạo ra yêu cầu cần có một khối trung gian để chuyển đổi giao dịch. Cầu chuyển đổi AHB – APB có nhiệm vụ tiếp nhận yêu cầu từ phía AHB, tạo chu kỳ truy cập tương ứng trên APB, đồng thời trả dữ liệu hoặc phản hồi trạng thái về lại phía Master. Đây là cơ sở để các phần tiếp theo trình bày chi tiết hơn về đặc điểm của AHB, APB và các yêu cầu cần xử lý trong quá trình thiết kế cầu chuyển đổi [7][8].

2.2 CÁC ĐẶC ĐIỂM CỦA GIAO THỨC AHB

2.2.1 Cấu trúc pha địa chỉ và pha dữ liệu

Một đặc điểm quan trọng của giao thức AHB là mỗi giao dịch được tách thành hai pha chính: pha địa chỉ và pha dữ liệu. Trong pha địa chỉ, Master đưa ra địa chỉ cần truy cập cùng các tín hiệu điều khiển như hướng truyền, loại truyền và kiểu burst. Các thông tin này cho phép Slave xác định giao dịch sắp thực hiện là đọc hay ghi, giao dịch đơn lẻ hay thuộc một chuỗi truyền liên tiếp. Sau đó, trong pha dữ liệu, dữ liệu thực sự được trao đổi giữa Master và Slave theo hướng đã được xác định ở pha trước.

Khác với các bus đơn giản thường thực hiện tuần tự theo thứ tự phát địa chỉ, chờ xử lý rồi mới truyền dữ liệu, AHB cho phép pha địa chỉ của giao dịch sau chồng lấp với pha dữ liệu của giao dịch trước. Cơ chế này giúp tăng thông lượng truyền dữ liệu mà không nhất thiết phải tăng tần số xung nhịp.

- *Pha địa chỉ*

Pha địa chỉ thường diễn ra trong một chu kỳ xung nhịp. Trong giai đoạn này, Master đưa ra đồng thời các tín hiệu điều khiển cần thiết cho giao dịch, bao gồm:

- HADDR[31:0]: địa chỉ ô nhớ hoặc thanh ghi ngoại vi cần truy cập.
- HWRITE: xác định hướng giao dịch, mức cao dùng cho thao tác ghi và mức thấp dùng cho thao tác đọc.
- HBURST[2:0]: xác định kiểu truyền burst nếu giao dịch thuộc một chuỗi truyền liên tiếp.
- HTRANS[1:0]: cho biết trạng thái truyền hiện tại và xác định chu kỳ đó có phải là một yêu cầu hợp lệ hay không.

Các tín hiệu này được Master thiết lập ổn định trong pha địa chỉ và được Slave lấy mẫu tại cạnh lên của xung nhịp. Nhờ đó, Slave có đủ thông tin để chuẩn bị cho pha dữ liệu diễn ra ở chu kỳ tiếp theo.

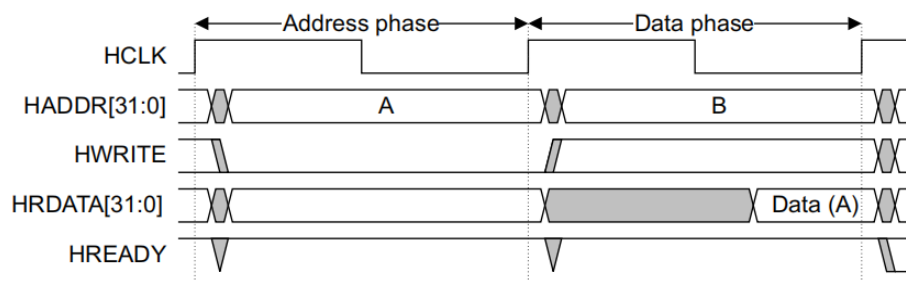
- *Pha dữ liệu*

Sau pha địa chỉ là pha dữ liệu, nơi dữ liệu thực sự được trao đổi giữa Master và Slave. Cách thức truyền dữ liệu phụ thuộc vào hướng giao dịch:

- Với giao dịch đọc, sau khi nhận địa chỉ và tín hiệu điều khiển, Slave được chọn thực hiện thao tác đọc và đưa dữ liệu trả về lên đường HRDATA[31:0]. Master chỉ lấy dữ liệu khi tín hiệu HREADY ở mức cao, cho biết dữ liệu trên bus đã hợp lệ.
- Với giao dịch ghi, Master đặt HWRITE = 1 trong pha địa chỉ. Sang pha dữ liệu, Master đưa dữ liệu cần ghi lên đường HWDATA[31:0]. Slave tiếp nhận dữ liệu này khi HREADY ở mức cao.

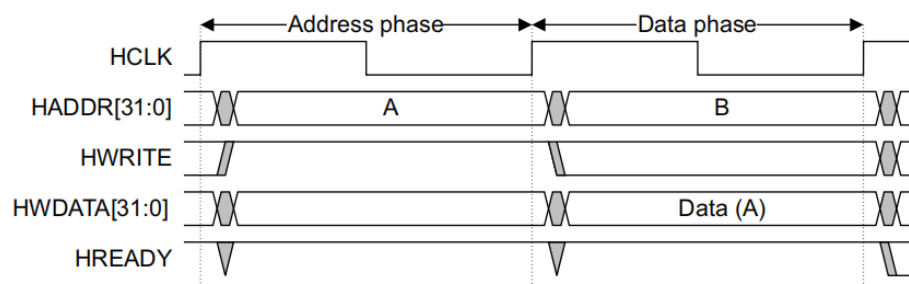
Trong trường hợp Slave cần thêm thời gian xử lý, pha dữ liệu có thể kéo dài hơn một chu kỳ xung nhịp. Khi đó, Slave giữ HREADY = 0 để yêu cầu Master chờ, đồng thời duy trì các tín hiệu liên quan cho đến khi giao dịch hoàn tất. Khi HREADY trở lại mức cao, giao dịch được xem là hoàn thành và bus có thể tiếp tục xử lý giao dịch kế tiếp.

Hình 2.1 và hình 2.2 minh họa dạng sóng của hai trường hợp giao dịch đọc và ghi trên AHB.



Hình 2.1: Giải đồ dạng sóng của giao dịch đọc AHB [1]

Trong giao dịch đọc ở hình 2.1, Master thiết lập HWRITE = 0 cùng với địa chỉ trên HADDR trong pha địa chỉ. Sang pha dữ liệu, Slave trả dữ liệu về thông qua đường HRDATA. Dữ liệu chỉ được xem là hợp lệ khi HREADY ở mức cao.



Hình 2.2: Giải đồ dạng sóng của giao dịch ghi AHB [1]

Đối với giao dịch ghi ở hình 2.2, Master thiết lập HWRITE = 1 trong pha địa chỉ. Ở pha dữ liệu kế tiếp, Master đưa dữ liệu cần ghi lên đường HWDATA để Slave tiếp nhận. Tương tự giao dịch đọc, quá trình ghi chỉ hoàn tất khi HREADY ở mức cao.

Việc phân tách pha địa chỉ và pha dữ liệu giúp AHB hỗ trợ cơ chế truyền theo đường ống, trong đó nhiều giao dịch có thể được xử lý chồng lấp qua các chu kỳ liên tiếp. Đây cũng là cơ sở để thiết kế cầu chuyển đổi AHB–APB cần có cơ chế điều khiển phù hợp nhằm lưu giữ thông tin địa chỉ, điều khiển dữ liệu và xử lý trạng thái chờ giữa hai giao thức.

2.2.2 Trạng thái giao dịch và truyền burst

Bên cạnh pha địa chỉ và pha dữ liệu, giao thức AHB còn sử dụng tín hiệu HTRANS[1:0] để xác định trạng thái của từng chu kỳ truyền. Tín hiệu này giúp Slave nhận biết chu kỳ hiện tại có phải là một giao dịch hợp lệ hay chỉ là chu kỳ rỗi hoặc chu kỳ chờ trong quá trình truyền dữ liệu.

Các trạng thái giao dịch chính của AHB gồm IDLE, BUSY, NONSEQ và SEQ. Trong đó, IDLE cho biết bus không có giao dịch mới, còn BUSY được dùng khi Master tạm thời chưa sẵn sàng phát nhịp tiếp theo trong một chuỗi truyền. Trạng thái NONSEQ được sử dụng cho giao dịch đơn hoặc nhịp đầu tiên của một chuỗi burst, trong khi SEQ được dùng cho các nhịp tiếp theo trong cùng chuỗi truyền.

Bảng 2.1: Các trạng thái giao dịch của AHB

Trạng thái	HTRANS[1:0]	Mô tả
IDLE	00	Cho biết không có giao dịch nào diễn ra. Slave phải phản hồi OKAY và bỏ qua dữ liệu.
BUSY	01	Master đang trong cụm burst nhưng nhịp tiếp theo chưa sẵn sàng. Slave phải duy trì phản hồi OKAY.
NONSEQ	10	Dùng cho truyền đơn lẻ hoặc nhịp đầu tiên của một cụm burst.
SEQ	11	Các nhịp tiếp theo của cụm burst. Địa chỉ tăng dần từ nhịp trước.

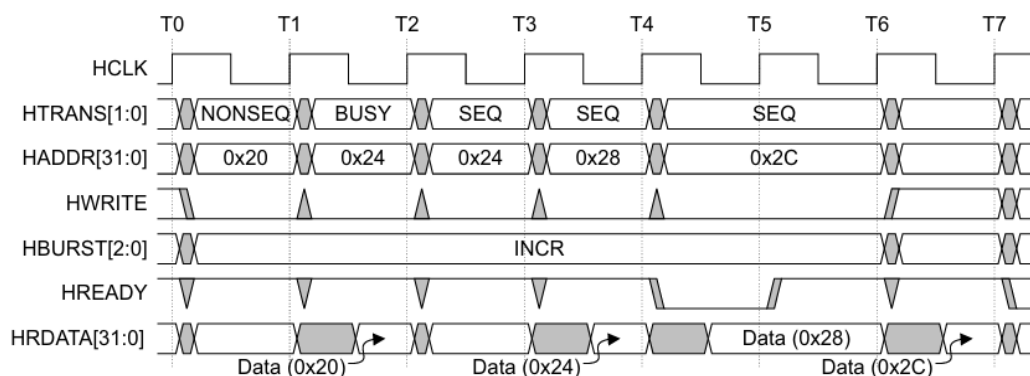
Đối với các giao dịch liên tiếp, AHB hỗ trợ cơ chế truyền burst. Thay vì thực hiện nhiều giao dịch đơn rời rạc, Master có thể phát một chuỗi các nhịp truyền có quan hệ với nhau. Điều này giúp giảm thời gian điều khiển bus và tăng hiệu quả truyền dữ liệu, đặc biệt trong các truy cập bộ nhớ hoặc truyền nhiều dữ liệu liên tiếp.

Các kiểu burst được xác định thông qua tín hiệu HBURST[2:0]. Trong phạm vi đề tài, các kiểu burst như SINGLE, INCR, INCR4, WRAP4, INCR8, WRAP8, INCR16 và WRAP16 được dùng để kiểm tra khả năng xử lý các chuỗi giao dịch của cầu chuyển đổi.

Bảng 2.2: Các kiểu truyền burst của AHB

HBURST[2:0]	Tên kiểu burst	Mô tả
000	SINGLE	Giao dịch đơn lẻ
001	INCR	Địa chỉ tăng liên tiếp, độ dài không xác định
010	WRAP4	Địa chỉ quay vòng trong 4 nhịp
011	INCR4	Địa chỉ tăng liên tiếp trong 4 nhịp
100	WRAP8	Địa chỉ quay vòng trong 8 nhịp
101	INCR8	Địa chỉ tăng liên tiếp trong 8 nhịp
110	WRAP16	Địa chỉ quay vòng trong 16 nhịp
111	INCR16	Địa chỉ tăng liên tiếp trong 16 nhịp

Giản đồ dạng sóng minh họa quá trình sử dụng HTRANS trong một chuỗi burst được trình bày ở hình 2.3.



Hình 2.3: Giản đồ dạng sóng các trạng thái giao dịch AHB [1]

Qua hình 2.3, có thể thấy nhịp đầu tiên của chuỗi burst sử dụng trạng thái NONSEQ, còn các nhịp tiếp theo sử dụng trạng thái SEQ. Nếu Master chưa sẵn sàng phát nhịp kế tiếp, trạng thái BUSY có thể được chèn vào mà không làm kết thúc chuỗi burst. Đặc điểm này yêu cầu cầu chuyển đổi AHB – APB phải nhận biết đúng trạng thái giao dịch, bỏ qua các chu kỳ không hợp lệ và chỉ chuyển tiếp các giao dịch cần xử lý sang phía APB.

2.2.3 Phản hồi giao dịch trên AHB

Trong giao thức AHB, sau khi Master phát yêu cầu đọc hoặc ghi, Slave cần trả về tín hiệu phản hồi để cho biết giao dịch đã hoàn tất hay có lỗi xảy ra. Hai tín hiệu quan trọng trong quá trình này là HREADY và HRESP.

Tín hiệu HREADY dùng để xác định trạng thái hoàn tất của giao dịch. Khi $HREADY = 1$, giao dịch hiện tại được chấp nhận và bus có thể chuyển sang giao dịch tiếp theo. Ngược lại, khi $HREADY = 0$, Slave yêu cầu Master giữ nguyên địa chỉ, dữ liệu và các tín hiệu điều khiển trên bus. Cơ chế này cho phép Slave kéo dài giao dịch khi cần thêm thời gian xử lý.

Tín hiệu HRESP dùng để báo trạng thái phản hồi của giao dịch. Khi giao dịch diễn ra bình thường, Slave trả về $HRESP = OKAY$. Nếu có lỗi, chẳng hạn như truy cập sai vùng địa chỉ hoặc ngoại vi trả về lỗi, Slave sẽ phát phản hồi $HRESP = ERROR$ để thông báo cho Master.

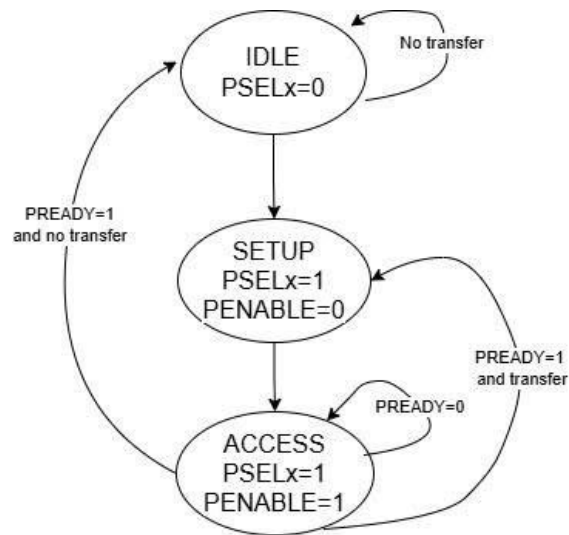
Đối với cầu chuyển đổi AHB – APB, cơ chế phản hồi này có vai trò quan trọng trong việc đồng bộ giữa hai bus. Do APB có thể phản hồi chậm thông qua PREADY, cầu chuyển đổi cần điều khiển HREADY để tạm dừng phía AHB cho đến khi giao dịch APB hoàn tất. Ngoài ra, nếu phía APB phát sinh lỗi thông qua PSLVERR, cầu chuyển đổi phải chuyển lỗi này về phía AHB bằng tín hiệu HRESP.

2.3 CÁC ĐẶC ĐIỂM CỦA GIAO THỨC APB

2.3.1 Chu kỳ SETUP và ACCESS

Giao thức APB thực hiện giao dịch theo trình tự đơn giản, không sử dụng cơ chế đường ống như AHB [2]. Mỗi giao dịch đọc hoặc ghi thường được chia thành hai pha chính là SETUP và ACCESS. Trong đó, pha SETUP dùng để thiết lập địa chỉ, hướng

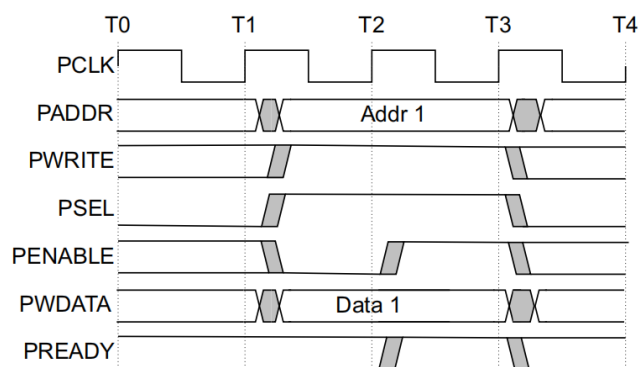
giao dịch và tín hiệu chọn Slave; pha ACCESS dùng để thực hiện truyền dữ liệu và hoàn tất giao dịch. Sơ đồ trạng thái của giao thức APB được trình bày ở hình 2.4.



Hình 2.4: Sơ đồ trạng thái của giao thức APB

Như hình 2.4 ở trạng thái IDLE, bus chưa có giao dịch nên PSEL và PENABLE đều ở mức thấp. Khi có yêu cầu truy cập, bus chuyển sang pha SETUP. Tại pha này, PSEL được kích hoạt để chọn Slave, đồng thời các tín hiệu PADDR, PWRITE và dữ liệu ghi nếu có được thiết lập ổn định. Sau đó, giao dịch chuyển sang pha ACCESS bằng cách đưa PENABLE lên mức cao. Nếu PREADY = 1, giao dịch kết thúc trong chu kỳ ACCESS; nếu PREADY = 0, pha ACCESS được kéo dài cho đến khi Slave sẵn sàng.

Dạng sóng của giao dịch ghi APB được minh họa ở hình 2.5.

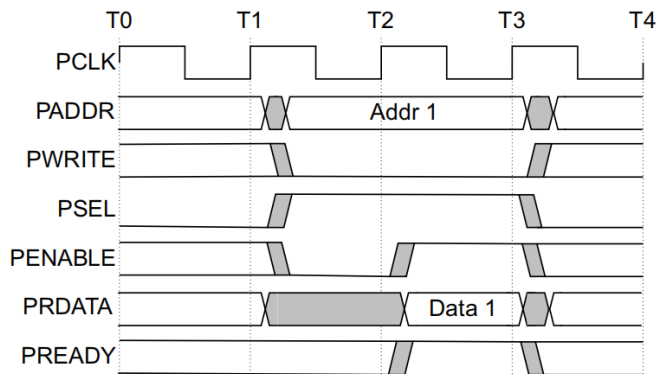


Hình 2.5: Giảm đồ dạng sóng của giao dịch ghi APB [2]

Trong giao dịch ghi ở hình 2.5, Master chọn Slave bằng PSEL, đưa địa chỉ lên PADDR, đặt PWRITE = 1 và phát dữ liệu ghi qua PWDATA trong pha SETUP. Sang

pha ACCESS, PENABLE được kích hoạt để Slave tiếp nhận dữ liệu. Khi PREADY ở mức cao, dữ liệu ghi được chấp nhận và giao dịch kết thúc.

Dạng sóng của giao dịch đọc APB được minh họa ở hình 2.6.



Hình 2.6: Giảm đồ dạng sóng của giao dịch đọc APB [2]

Đối với giao dịch đọc hình 2.6, Master cũng bắt đầu bằng cách kích hoạt PSEL, đưa địa chỉ cần đọc lên PADDR và đặt PWRITE = 0 trong pha SETUP. Khi chuyển sang pha ACCESS, PENABLE được đưa lên mức cao và Slave trả dữ liệu về qua PRDATA. Nếu PREADY = 1, dữ liệu đọc được xem là hợp lệ và giao dịch hoàn tất.

Như vậy, hai pha SETUP và ACCESS là cơ sở để cầu chuyển đổi AHB – APB tạo đúng trình tự điều khiển cho phía APB. Khi nhận được giao dịch từ AHB, cầu chuyển đổi cần thiết lập địa chỉ, dữ liệu và tín hiệu chọn Slave ở pha SETUP, sau đó kích hoạt PENABLE ở pha ACCESS để hoàn tất truy cập với ngoại vi APB.

2.3.2 Cơ chế chờ bằng PREADY

Trong giao thức APB, tín hiệu PREADY được Slave sử dụng để báo trạng thái hoàn tất giao dịch. Khi Slave sẵn sàng phản hồi, PREADY được đưa lên mức cao và giao dịch kết thúc trong pha ACCESS. Ngược lại, nếu Slave cần thêm thời gian xử lý, PREADY được giữ ở mức thấp để kéo dài pha ACCESS.

Trong thời gian PREADY = 0, các tín hiệu như PSEL, PENABLE, PADDR, PWRITE và dữ liệu liên quan phải được giữ ổn định. Điều này giúp Slave có đủ thời gian xử lý yêu cầu mà không làm sai lệch thông tin giao dịch. Khi PREADY chuyển lên mức cao, dữ liệu đọc hoặc ghi được chấp nhận và giao dịch APB được xem là hoàn tất.

Đối với cầu chuyển đổi AHB – APB, cơ chế chờ bằng PREADY có vai trò quan trọng trong việc đồng bộ tốc độ giữa hai bus. Vì APB Slave có thể phản hồi chậm hơn AHB Master, cầu chuyển đổi cần theo dõi PREADY để quyết định thời điểm kết thúc giao dịch APB. Trong lúc APB chưa hoàn tất, phía AHB sẽ được tạm dừng thông qua tín hiệu Hreadyout. Khi PREADY = 1, cầu chuyển đổi trả lại trạng thái sẵn sàng cho AHB Master để tiếp tục giao dịch tiếp theo.

2.3.3 Phản hồi lỗi bằng PSLVERR

Bên cạnh tín hiệu PREADY, giao thức APB còn sử dụng PSLVERR để báo lỗi trong quá trình truy cập ngoại vi. Tín hiệu này chỉ có ý nghĩa ở pha ACCESS, khi PSEL và PENABLE đều đang được kích hoạt. Nếu giao dịch hoàn tất bình thường, PSLVERR giữ ở mức thấp. Ngược lại, khi Slave phát hiện lỗi, PSLVERR được đưa lên mức cao để thông báo rằng giao dịch không thành công.

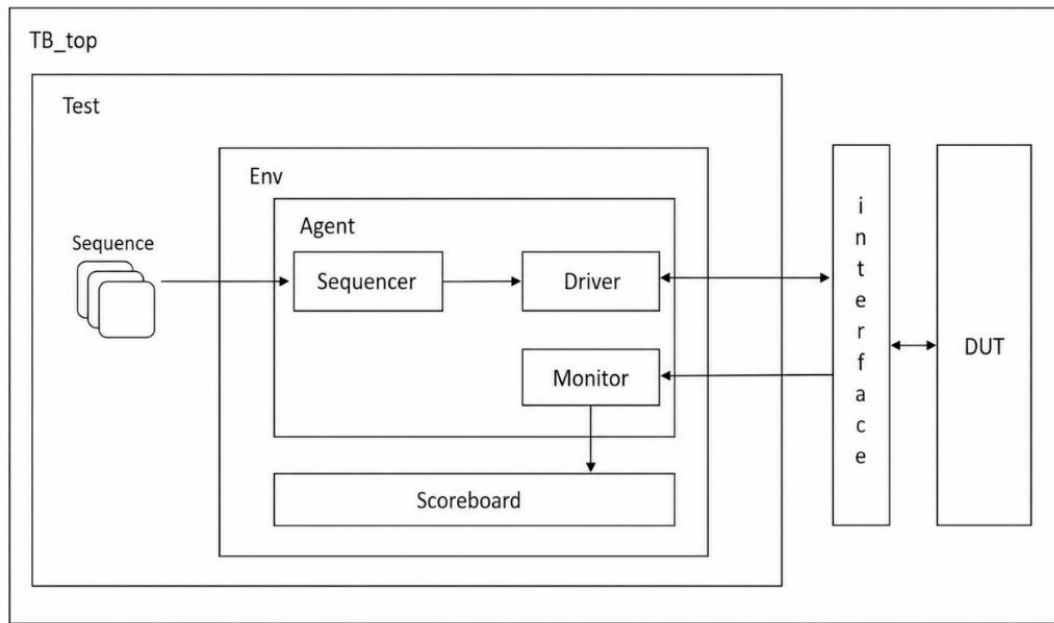
Các lỗi có thể phát sinh khi ngoại vi không hỗ trợ địa chỉ truy cập, không cho phép thao tác đọc hoặc ghi tại vùng đó, hoặc gặp lỗi nội bộ trong quá trình xử lý. Khi PREADY = 1 và PSLVERR = 1, giao dịch APB được xem là đã kết thúc nhưng có trạng thái lỗi.

Đối với cầu chuyển đổi AHB – APB, tín hiệu PSLVERR cần được chuyển đổi thành phản hồi lỗi ở phía AHB. Khi phát hiện lỗi từ APB Slave, cầu chuyển đổi phải tạo phản hồi HRESP = ERROR để báo cho AHB Master biết giao dịch không hợp lệ. Đồng thời, quá trình phản hồi phải được điều khiển đúng với cơ chế của AHB để Master có thể kết thúc giao dịch lỗi và tiếp tục hoạt động bình thường.

2.4 CƠ SỞ LÝ THUYẾT VỀ MÔI TRƯỜNG UVM

2.4.1 Mô hình kiểm thử theo giao dịch

Trong UVM, quá trình kiểm thử được mô tả ở mức giao dịch thay vì điều khiển trực tiếp từng tín hiệu riêng lẻ [4][6]. Mỗi giao dịch được đóng gói thành một đối tượng chứa các thông tin cần thiết như địa chỉ, dữ liệu, hướng truyền, loại giao dịch và trạng thái phản hồi. Cách tổ chức này giúp môi trường kiểm thử dễ quản lý hơn, đặc biệt với các thiết kế có nhiều loại giao dịch như cầu chuyển đổi AHB – APB. Kiến trúc tổng quát của một môi trường UVM được minh họa ở hình 2.7.



Hình 2.7: Kiến trúc tổng quát của môi trường UVM

Dựa trên hình 2.7, lớp Test là thành phần cấp cao nhất, có nhiệm vụ thiết lập kịch bản kiểm thử và cấu hình môi trường. Bên dưới là Environment, nơi quản lý các thành phần xác minh như Agent và Scoreboard. Mỗi Agent thường gồm Sequencer, Driver và Monitor. Sequencer cấp phát các giao dịch kiểm thử cho Driver, Driver chuyển các giao dịch này thành tín hiệu để đưa vào DUT, còn Monitor quan sát tín hiệu từ DUT và đóng gói lại thành giao dịch.

2.4.2 Vai trò của các thành phần trong môi trường UVM

Các thành phần xác minh được tổ chức theo từng vai trò riêng để quá trình kiểm thử rõ ràng, dễ quản lý và dễ mở rộng. Một số thành phần quan trọng thường gặp gồm Agent, Driver, Monitor và Scoreboard.

Agent là thành phần đại diện cho một giao diện kiểm thử trong môi trường UVM. Bên trong Agent thường có Sequencer, Driver và Monitor. Tùy theo mục đích sử dụng, Agent có thể hoạt động ở chế độ chủ động để tạo kích thích cho DUT, hoặc ở chế độ thụ động để chỉ quan sát tín hiệu.

Driver có nhiệm vụ nhận các giao dịch từ Sequencer và chuyển chúng thành tín hiệu cụ thể trên giao diện kết nối với DUT. Đây là thành phần trực tiếp điều khiển tín hiệu đầu vào của thiết kế theo nội dung giao dịch đã được tạo ra.

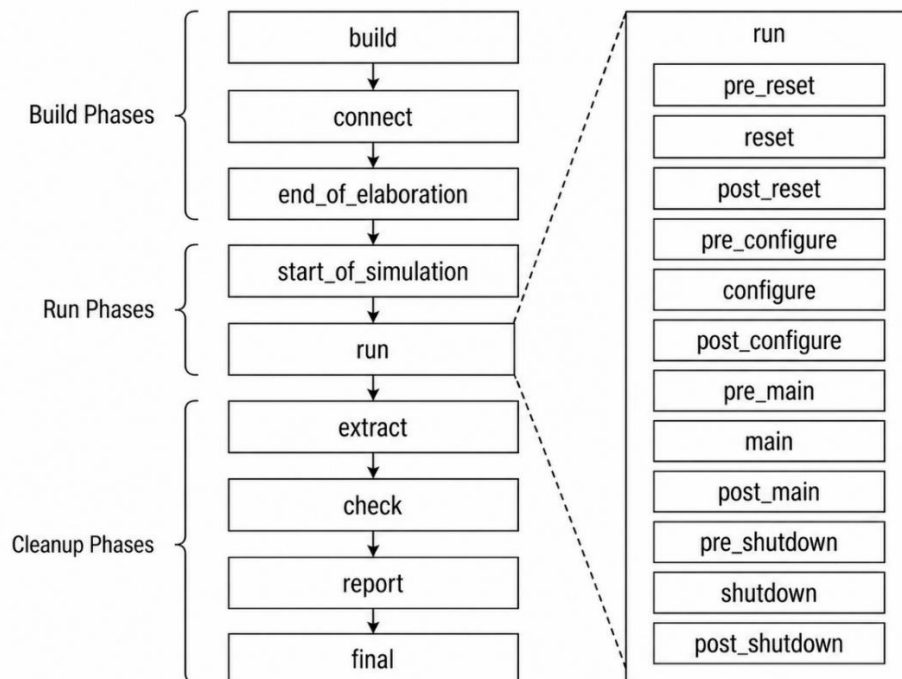
Monitor hoạt động thụ động, chỉ theo dõi các tín hiệu trên giao diện mà không làm thay đổi hoạt động của DUT. Các tín hiệu quan sát được sẽ được Monitor đóng gói lại thành giao dịch và gửi đến các thành phần khác trong môi trường kiểm thử.

Scoreboard dùng để kiểm tra kết quả mô phỏng. Thành phần này nhận dữ liệu từ Monitor, sau đó so sánh kết quả thực tế với kết quả mong đợi. Nếu hai kết quả khớp nhau, giao dịch được xem là đúng; nếu có sai lệch, Scoreboard sẽ ghi nhận lỗi để phục vụ quá trình phân tích.

Nhờ cách phân chia vai trò này, môi trường UVM có cấu trúc rõ ràng hơn. Mỗi thành phần chỉ tập trung vào một nhiệm vụ cụ thể, giúp việc xây dựng, mở rộng và tái sử dụng môi trường kiểm thử trở nên thuận tiện hơn.

2.4.3 Cơ chế pha trong UVM

Trong UVM, quá trình mô phỏng được chia thành nhiều pha thực thi khác nhau. Mỗi pha đảm nhiệm một nhiệm vụ riêng, giúp các thành phần trong môi trường kiểm thử được tạo, kết nối, cấu hình và vận hành theo đúng thứ tự. Cơ chế này giúp tránh tình trạng các thành phần hoạt động chưa đồng bộ hoặc chưa được khởi tạo đầy đủ trước khi mô phỏng bắt đầu. Trình tự các pha chính trong UVM được minh họa ở hình 2.8.



Hình 2.8: Trình tự thực thi các pha trong UVM [6]

Dựa trên hình 2.8, các pha đầu tiên như `build_phase`, `connect_phase` và `end_of_elaboration_phase` được dùng để xây dựng cấu trúc môi trường, kết nối các thành phần và kiểm tra lại hệ thống trước khi chạy mô phỏng. Trong đó, `build_phase` tạo các thành phần cần thiết, còn `connect_phase` thiết lập đường truyền dữ liệu giữa các khối như Driver, Monitor và Scoreboard.

Sau giai đoạn xây dựng, UVM chuyển sang các pha chuẩn bị như `start_of_simulation_phase` để hoàn tất cấu hình trước khi mô phỏng chính thức bắt đầu. Pha quan trọng nhất là `run_phase`, nơi các kích thích kiểm thử được tạo ra, tín hiệu được điều khiển vào DUT và dữ liệu mô phỏng được thu thập.

Khi quá trình mô phỏng kết thúc, các pha như `extract_phase`, `check_phase`, `report_phase` và `final_phase` được thực hiện. Những pha này dùng để trích xuất dữ liệu, kiểm tra kết quả, tổng hợp báo cáo và kết thúc mô phỏng. Nhờ cơ chế phân pha, môi trường UVM có trình tự hoạt động rõ ràng, giúp quá trình kiểm thử dễ kiểm soát và dễ mở rộng hơn.

Chương 3

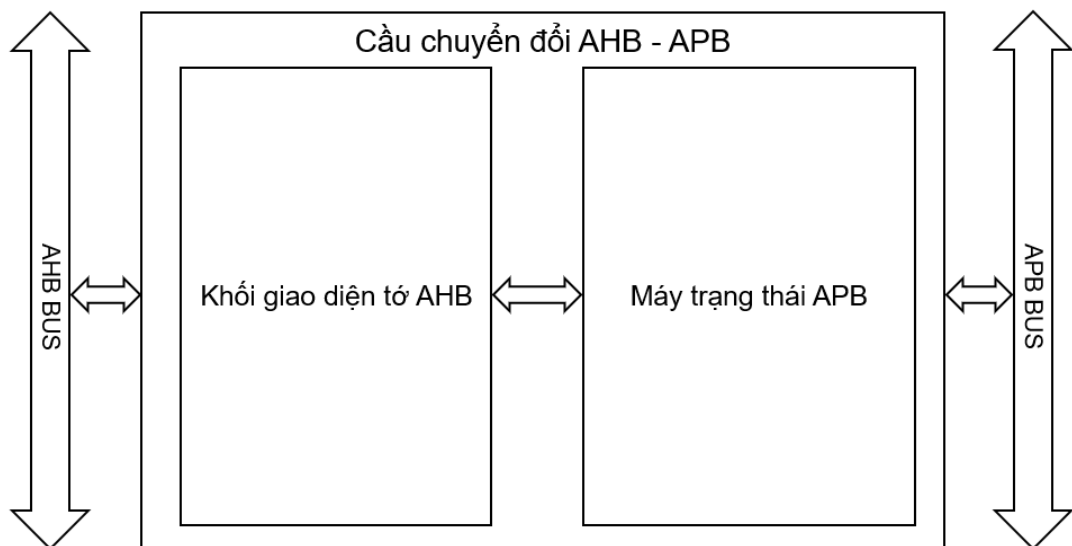
THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU CỦA HỆ THỐNG

Hệ thống được thiết kế nhằm xây dựng cầu chuyển đổi giữa hai giao thức AHB và APB, cho phép một AHB Master giao tiếp với tối đa bốn APB Slave. Để xử lý trường hợp các ngoại vi APB phản hồi chậm, thiết kế tích hợp cơ chế tạm dừng, giúp duy trì sự đồng bộ cho đến khi quá trình truyền hoàn tất. Ngoài ra, hệ thống cũng hỗ trợ phát hiện lỗi từ phía APB và phản hồi trực tiếp về AHB Master. Về kiểm tra chức năng, thiết kế được mô phỏng và xác minh bằng môi trường UVM. Sau đó, hệ thống được tổng hợp trên FPGA bằng công cụ Vivado nhằm đánh giá tài nguyên sử dụng, tần số hoạt động và công suất tiêu thụ.

3.2 SƠ ĐỒ KHỐI HỆ THỐNG

Cầu chuyển đổi AHB – APB được thiết kế theo mô hình gồm các thành phần đảm nhiệm việc tiếp nhận, xử lý và chuyển đổi tín hiệu giữa hai giao thức AHB và APB. Sơ đồ khối tổng quát của hệ thống được trình bày trong hình 3.1.

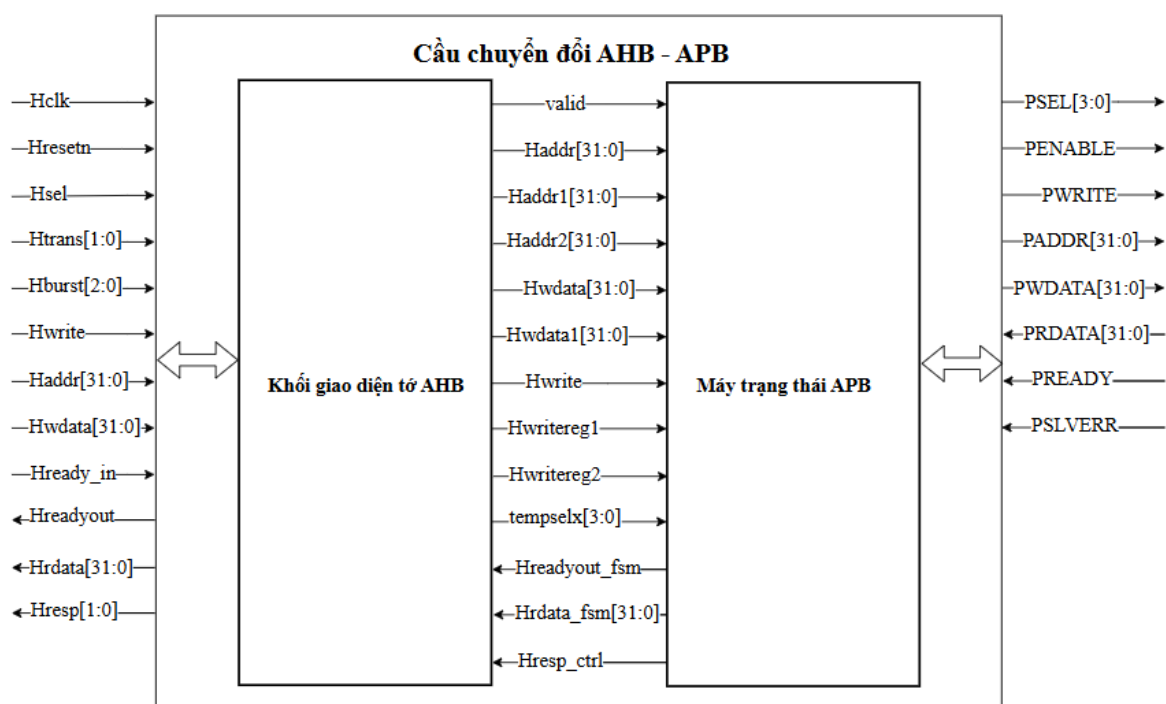


Hình 3.1: Sơ đồ khối tổng quát cầu chuyển đổi AHB – APB

Dựa trên sơ đồ khối của hệ thống ở hình trên, các khối chính có chức năng cụ thể như sau:

- Khối giao diện tớ AHB: Khối này đảm nhiệm việc giao tiếp trực tiếp với AHB Master, tiếp nhận các yêu cầu truy cập, kiểm tra tính hợp lệ của tín hiệu và xử lý cơ chế truyền dữ liệu đường ống của chuẩn giao thức AHB. Bên cạnh đó, trực tiếp thực hiện việc giải mã không gian bộ nhớ để định tuyến đến đúng APB Slave. Dựa trên kết quả giải mã và trạng thái xử lý để tự động điều phối và gửi các tín hiệu phản hồi trạng thái ngược trở lại cho AHB Master.
- Máy trạng thái APB: Là máy trạng thái hữu hạn hoạt động để quản lý chu kỳ của các giao dịch trên bus APB. Khối tiếp nhận thông tin từ cơ chế đường ống của AHB nhằm điều phối các tín hiệu điều khiển ngoại vi. Ngoài ra, khối còn có chức năng giám sát các tín hiệu phản hồi từ phía APB Slave để xử lý các tình huống chờ hoặc lỗi phát sinh trong quá trình trao đổi dữ liệu.
- Cầu chuyển đổi AHB – APB: Là mô-đun mức cao nhất, có nhiệm vụ tích hợp hai khối trên thành một hệ thống hoàn chỉnh và định tuyến các tín hiệu nội bộ ra cổng giao tiếp chuẩn để kết nối trực tiếp với hệ thống.

Để có cái nhìn tổng quan về cách cầu chuyển đổi AHB – APB giao tiếp với các thành phần bên ngoài, sơ đồ các cổng tín hiệu của hệ thống được mô tả dưới hình 3.2.



Hình 3.2: Các tín hiệu đầu vào và đầu ra của cầu chuyển đổi AHB – APB

Như ở hình 3.2 trên, chức năng cụ thể của từng tín hiệu đầu vào và đầu ra kết nối với AHB Master và APB Slave được mô tả chi tiết trong bảng 3.1.

Bảng 3.1: Tín hiệu đầu vào và đầu ra của hệ thống

STT	Tín hiệu	Số bit	I/O	Mô tả
1	Hclk	1	I	Xung nhịp hệ thống dùng chung cho AHB và APB
2	Hresetn	1	I	Tín hiệu đặt lại toàn hệ thống, tích cực mức thấp
3	Hsel	1	I	Tín hiệu chọn Slave từ bộ giải mã địa chỉ hệ thống
4	Htrans	2	I	Loại giao dịch: IDLE, BUSY, NONSEQ, SEQ
5	Hburst	3	I	Kiểu burst AHB: SINGLE, INCR, WRAP4, INCR4,...
6	Hwrite	1	I	Hướng giao dịch: 1 = ghi, 0 = đọc
7	Haddr	32	I	Địa chỉ AHB
8	Hwdata	32	I	Dữ liệu ghi AHB
9	Hready_in	1	I	Tín hiệu báo hoàn tất giao dịch trước đó
10	Hreadyout	1	O	Tín hiệu hệ thống sẵn sàng nhận giao dịch tiếp theo
11	Hrdata	32	O	Dữ liệu đọc trả về AHB Master
12	Hresp	2	O	Phản hồi AHB: OKAY (00) hoặc ERROR (01)
13	PSEL	4	O	Tín hiệu chọn ngoại vi
14	PENABLE	1	O	Tín hiệu sẵn sàng pha ENABLE của giao tiếp APB
15	PWRITE	1	O	Hướng giao dịch APB: 1 = ghi, 0 = đọc
16	PADDR	32	O	Địa chỉ APB gửi đến Slave
17	PWDATA	32	O	Dữ liệu ghi APB
18	PRDATA	32	I	Dữ liệu đọc trả về từ APB Slave
19	PREADY	1	I	Tín hiệu sẵn sàng từ APB Slave
20	PSLVERR	1	I	Tín hiệu lỗi từ APB Slave

Trong thiết kế này, cầu chuyển đổi AHB – APB hoạt động trong cùng một miền xung nhịp. Do đó, phía APB không sử dụng tín hiệu PCLK riêng mà dùng chung xung nhịp Hclk của hệ thống.

Ngoài các tín hiệu giao tiếp bên ngoài, quá trình chuyển đổi giữa hai giao thức còn được thực hiện thông qua các tín hiệu nội bộ giữa khối giao diện tớ AHB và máy trạng thái APB. Chức năng và hướng truyền của các tín hiệu này được trình bày trong bảng 3.2.

Bảng 3.2: Tín hiệu nội bộ giữa khối giao diện tớ AHB và máy trạng thái APB

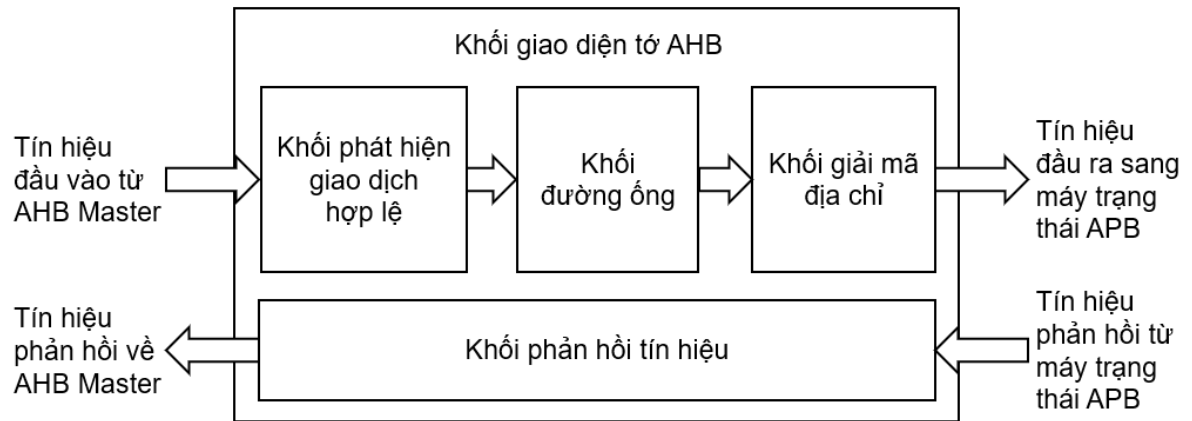
STT	Tín hiệu	Số bit	Chiều	Mô tả
1	valid	1	AHB → FSM	Cờ báo hiệu có một giao dịch AHB hợp lệ đang diễn ra để báo cho FSM
2	Haddr	32	AHB → FSM	Tín hiệu địa chỉ
3	Haddr1	32	AHB → FSM	Tín hiệu địa chỉ trễ 1 chu kỳ
4	Haddr2	32	AHB → FSM	Tín hiệu địa chỉ trễ 2 chu kỳ
5	Hwdata	32	AHB → FSM	Dữ liệu ghi
6	Hwdata1	32	AHB → FSM	Dữ liệu ghi trễ 1 chu kỳ
7	Hwrite	1	AHB → FSM	Tín hiệu hướng giao dịch
8	Hwritereg1	1	AHB → FSM	Tín hiệu hướng giao dịch trễ 1 chu kỳ
9	Hwritereg2	1	AHB → FSM	Tín hiệu hướng giao dịch trễ 2 chu kỳ
10	tempselx	4	AHB → FSM	Tín hiệu chỉ định chọn ngoại vi APB
11	Hreadyout_fsm	1	FSM → AHB	Tín hiệu trạng thái sẵn sàng
12	Hrdata_fsm	32	FSM → AHB	Dữ liệu đọc của ngoại vi
13	Hresp_ctrl	1	FSM → AHB	Cờ báo lỗi từ ngoại vi

Bảng 3.2 cho thấy các tín hiệu nội bộ chủ yếu truyền từ khối giao diện tớ AHB sang máy trạng thái APB để cung cấp thông tin điều khiển giao dịch. Chiều ngược lại chỉ gồm các tín hiệu phản hồi như trạng thái hoàn tất, dữ liệu đọc và lỗi, giúp luồng xử lý giữa hai khối được rõ ràng và dễ kiểm soát.

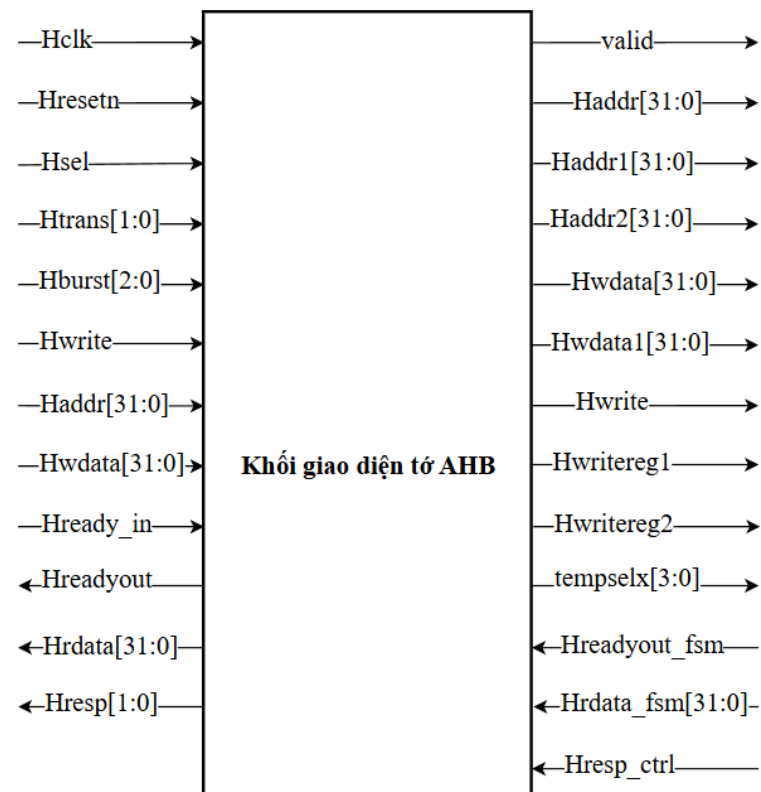
3.3 THIẾT KẾ CHI TIẾT KHỐI

3.3.1 Thiết kế khối giao diện tứ AHB

Khối giao diện tứ AHB là nơi tiếp nhận và xử lý các yêu cầu truy cập từ bus AHB. Kiến trúc bên trong của khối được trình bày ở hình 3.3, còn các tín hiệu vào ra được thể hiện ở hình 3.4.



Hình 3.3: Kiến trúc nội bộ của khối giao diện tứ AHB

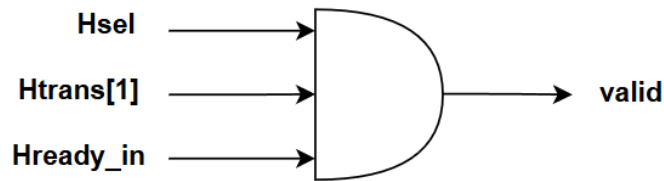


Hình 3.4: Các tín hiệu đầu vào và đầu ra của khối giao diện tứ AHB

Hình 3.3 cho thấy khối giao diện tớ AHB được chia thành bốn phần chính: đường ống lưu trữ tín hiệu AHB, khối kiểm tra giao dịch hợp lệ, khối giải mã địa chỉ chọn ngoại vi APB và khối tạo tín hiệu phản hồi về Master.

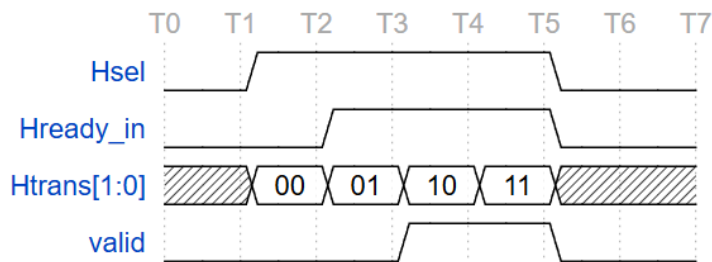
3.3.1.1 Thiết kế khối phát hiện giao dịch hợp lệ

Khối phát hiện giao dịch hợp lệ dùng để xác định thời điểm yêu cầu từ AHB Master đủ điều kiện xử lý. Khi giao dịch hợp lệ, tín hiệu valid được kích hoạt để báo cho máy trạng thái APB bắt đầu thực hiện giao dịch. Sơ đồ logic của khối này được trình bày ở hình 3.5.



Hình 3.5: Sơ đồ logic khối phát hiện giao dịch hợp lệ

Dựa trên hình 3.5, tín hiệu valid chỉ được đưa lên mức logic cao khi cả ba tín hiệu Hsel, Htrans[1] và Hready_in cùng thỏa mãn điều kiện. Cụ thể, Hsel ở mức cao cho biết cầu chuyển đổi đang được chọn để thực hiện giao tiếp. Tín hiệu Htrans[1] dùng để nhận biết giao dịch hợp lệ, gồm NONSEQ hoặc SEQ, đồng thời loại bỏ các chu kỳ IDLE và BUSY. Trong khi đó, Hready_in xác nhận bus đã sẵn sàng và giao dịch trước đó đã hoàn tất. Khi các điều kiện này đồng thời đúng, khối sẽ kích hoạt valid để báo cho máy trạng thái APB bắt đầu xử lý giao dịch. Hoạt động khối phát hiện giao dịch hợp lệ được minh họa thông qua giản đồ dạng sóng ở hình 3.6.



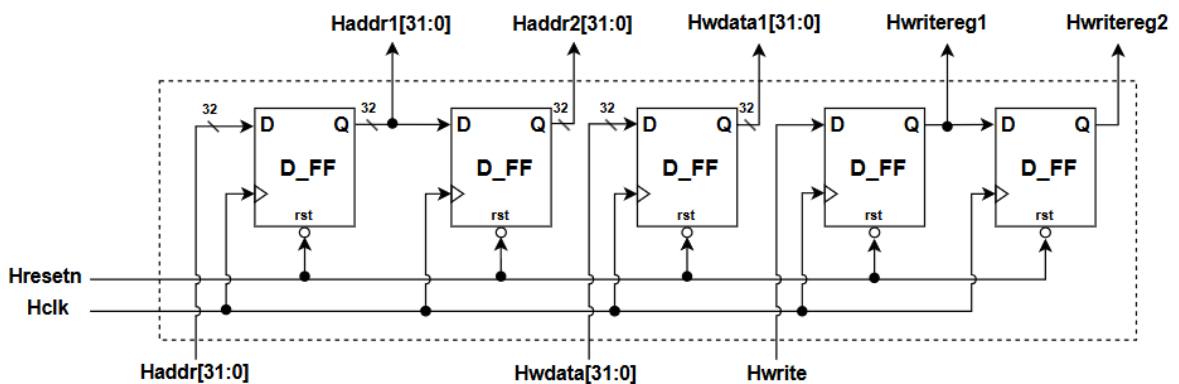
Hình 3.6: Giản đồ dạng sóng khối phát hiện giao dịch hợp lệ

Giản đồ dạng sóng ở hình 3.6 minh họa hoạt động của khối qua các mốc thời gian như sau:

- Từ T1 đến T2, mặc dù Hsel được kích hoạt ở mức 1 nhưng Hready_in vẫn ở mức thấp và bus đang ở trạng thái rỗng với Htrans = 00 tương ứng IDLE. Do bit Htrans[1] bằng 0 nên tín hiệu valid chưa được kích hoạt.
- Từ T2 đến T3, Htrans chuyển sang 01, tương ứng trạng thái BUSY. Ở trạng thái này, Htrans[1] vẫn bằng 0 nên điều kiện tạo giao dịch hợp lệ chưa thỏa mãn, vì vậy valid tiếp tục giữ mức thấp.
- Từ T3 đến T5, bus chuyển sang các trạng thái giao dịch hợp lệ gồm 10 tương ứng NONSEQ và 11 tương ứng SEQ. Khi đó Hsel, Htrans[1] và Hready_in đều thỏa mãn, nên tín hiệu valid được đưa lên mức logic cao để báo hiệu giao dịch hợp lệ.

3.3.1.2 Thiết kế khối đường ống

Khối đường ống có nhiệm vụ đồng bộ các tín hiệu trong quá trình truyền dữ liệu AHB. Do giao thức AHB tách giao dịch thành pha địa chỉ và pha dữ liệu lệch nhau một chu kỳ xung nhịp, các tín hiệu địa chỉ và điều khiển cần được lưu tạm và làm trễ tương ứng. Nhờ đó, địa chỉ, tín hiệu điều khiển và dữ liệu ghi được đưa đến máy trạng thái APB tại đúng thời điểm, giúp quá trình xử lý giao dịch với ngoại vi APB diễn ra ổn định và chính xác. Sơ đồ logic của khối đường ống được mô tả ở hình 3.7.

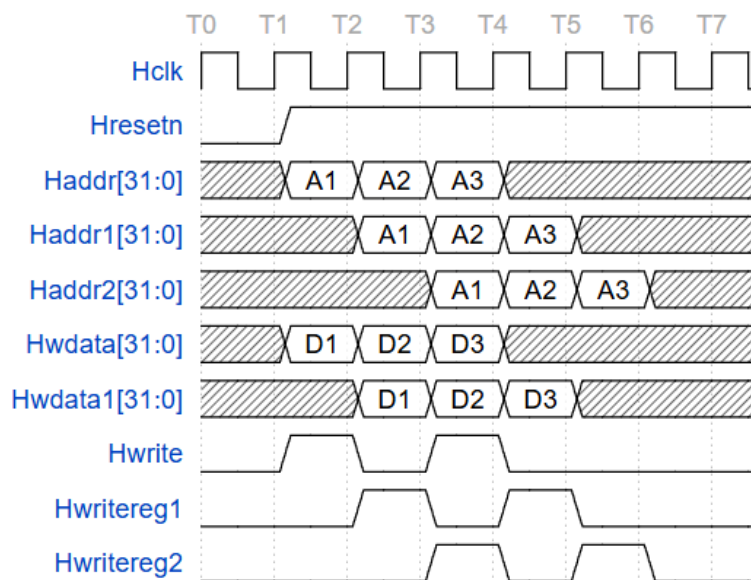


Hình 3.7: Sơ đồ logic khối đường ống

Cụ thể, khối đường ống thực hiện việc lưu trữ và tạo trễ các tín hiệu điều khiển thông qua hệ thống thanh ghi ở hình 3.7 như sau:

- Haddr[31:0] là tín hiệu địa chỉ từ AHB Master, được đưa qua hai tầng thanh ghi để tạo ra Haddr1[31:0] trễ một chu kỳ và Haddr2[31:0] trễ hai chu kỳ. Cách này giúp lưu giữ địa chỉ của giao dịch AHB trong các chu kỳ xử lý tiếp theo, phù hợp với quá trình truy cập chậm hơn của bus APB.
- Hwdata[31:0] là dữ liệu ghi từ Master, được lưu qua một tầng thanh ghi để tạo ra Hwdata1[31:0] trễ một chu kỳ. Tín hiệu này được dùng để cung cấp dữ liệu ghi đồng bộ cho máy trạng thái APB.
- Hwrite cho biết loại giao dịch là đọc hay ghi. Tín hiệu này được đưa qua hai tầng thanh ghi nối tiếp để tạo ra Hwritereg1 và Hwritereg2, tương ứng với độ trễ một và hai chu kỳ. Nhờ đó, hướng giao dịch được duy trì ổn định và đồng bộ với các pha xử lý của APB.

Sự dịch chuyển và làm trễ tín hiệu qua các tầng thanh ghi của khối đường ống được minh họa qua giản đồ dạng sóng ở hình 3.8.

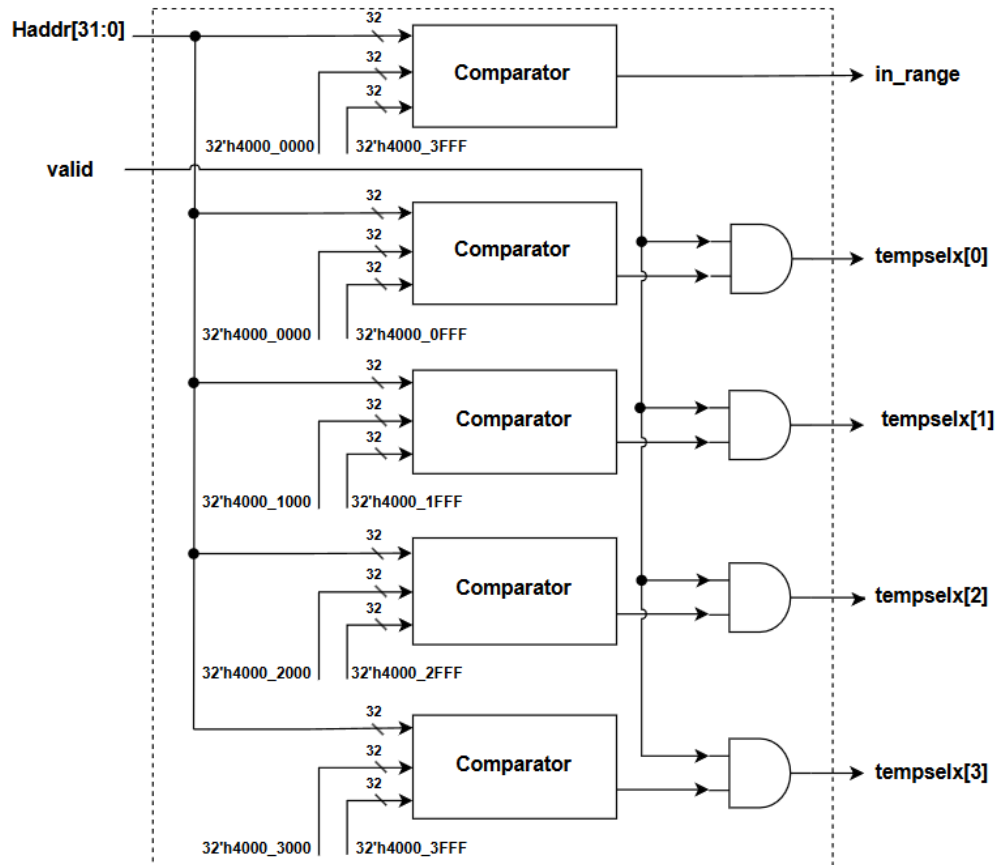


Hình 3.8: Giản đồ dạng sóng khối đường ống

Giản đồ dạng sóng ở hình 3.8 minh họa cơ chế tạo trễ của khối đường ống. Tại mỗi sườn lên của xung nhịp Hclk, các tín hiệu đầu vào như Haddr, Hwdata và Hwrite được ghi nhận và lần lượt đưa qua các tầng thanh ghi. Vì vậy, sau một chu kỳ sẽ xuất hiện các tín hiệu trễ như Haddr1, Hwdata1, Hwritereg1, và sau hai chu kỳ sẽ tạo ra Haddr2, Hwritereg2. Cách dịch chuyển tuần tự này giúp giữ lại thông tin của giao dịch AHB để sử dụng đúng thời điểm trong quá trình xử lý phía APB.

3.3.1.3 Thiết kế khối giải mã địa chỉ

Khối giải mã địa chỉ kiểm tra giá trị Haddr để xác định APB Slave cần truy cập. Đồng thời, khối này cũng dùng để nhận biết các địa chỉ không hợp lệ, với vùng địa chỉ được hỗ trợ nằm trong khoảng từ 32'h4000_0000 đến 32'h4000_3FFF. Sơ đồ logic của khối giải mã được mô tả ở hình 3.9 dưới đây:

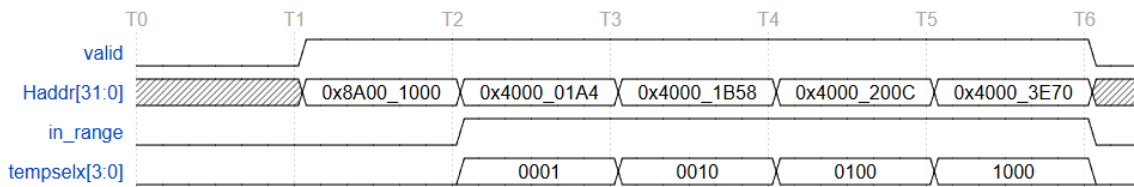


Hình 3.9: Sơ đồ logic của khối giải mã địa chỉ

Ở sơ đồ hình 3.9, khối giải mã địa chỉ tiến hành đối chiếu trực tiếp toàn bộ giá trị của bus địa chỉ 32-bit:

- Kiểm tra vùng địa chỉ: Khối giải mã kiểm tra địa chỉ Haddr[31:0] để xác định địa chỉ có nằm trong vùng nhớ hợp lệ hay không. Nếu địa chỉ thuộc dải được hỗ trợ, tín hiệu in_range sẽ được kích hoạt.
- Giải mã chọn Slave: Khi giao dịch hợp lệ, khối giải mã xác định địa chỉ đang truy cập thuộc vùng nhớ của APB Slave nào. Từ đó, tín hiệu tempselex được tạo ra để chọn đúng Slave tương ứng trong hệ thống.

Giản đồ dạng sóng thể hiện chức năng phân vùng và định tuyến địa chỉ của khối giải mã được thể hiện tại hình 3.10.



Hình 3.10: Giản đồ dạng sóng của khối giải mã địa chỉ

Giản đồ dạng sóng ở hình 3.10 minh họa nguyên lý hoạt động của khối giải mã địa chỉ, hoạt động chính của mạch được mô tả như sau:

- Từ T1 đến T2, khi giao dịch bắt đầu, tín hiệu valid được kích hoạt và khối giải mã bắt đầu kiểm tra địa chỉ Haddr[31:0]. Lúc này, Haddr = 32'h8A00_1000, nằm ngoài vùng địa chỉ hợp lệ của hệ thống nên tín hiệu in_range vẫn giữ ở mức thấp.
- Từ T2 đến T6, Haddr[31:0] lần lượt nhận các giá trị thuộc vùng địa chỉ hợp lệ. Vì vậy, tín hiệu in_range được đưa lên mức cao và khối giải mã bắt đầu xác định APB Slave tương ứng thông qua tín hiệu tempselx[3:0].
 - Từ T2 đến T3, địa chỉ 32'h4000_01A4 thuộc vùng của Slave 0 nên tempselx = 4'b0001.
 - Từ T3 đến T4, địa chỉ 32'h4000_1B58 thuộc vùng của Slave 1 nên tempselx = 4'b0010.
 - Từ T4 đến T5, địa chỉ 32'h4000_200C thuộc vùng của Slave 2 nên tempselx = 4'b0100.
 - Từ T5 đến T6, địa chỉ 32'h4000_3E70 thuộc vùng của Slave 3 nên tempselx = 4'b1000.

Chi tiết về quy hoạch dải địa chỉ bộ nhớ và giá trị của các tín hiệu giải mã tương ứng cho từng trạng thái hoạt động được tóm tắt trong bảng 3.3.

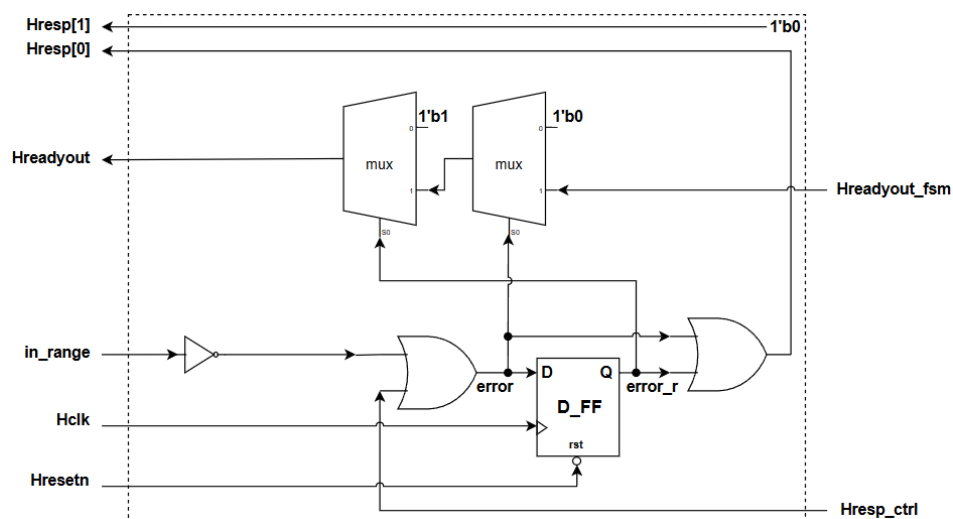
Bảng 3.3: Phân bố không gian địa chỉ và các tín hiệu giải mã chọn ngoại vi APB

APB Slave	Dải địa chỉ truy cập	in_range	tempselex[3:0]
Slave 0	0x4000_0000 – 0x4000_0FFF	1	4'b0001
Slave 1	0x4000_1000 – 0x4000_1FFF	1	4'b0010
Slave 2	0x4000_2000 – 0x4000_2FFF	1	4'b0100
Slave 3	0x4000_3000 – 0x4000_3FFF	1	4'b1000
Ngoài vùng nhớ	Nằm ngoài dải 0x4000_0000 – 0x4000_3FFF	0	4'b0000

Bảng 3.3 trình bày không gian địa chỉ APB được chia thành bốn vùng, tương ứng với bốn APB Slave. Mỗi vùng có kích thước 4KB và được lựa chọn thông qua tín hiệu tempselex[3:0]. Khi địa chỉ nằm trong vùng hợp lệ, tín hiệu in_range được đưa lên mức 1 hoặc nếu địa chỉ nằm ngoài vùng cho phép, in_range sẽ giữ ở mức 0.

3.3.1.4 Thiết kế khối phản hồi tín hiệu

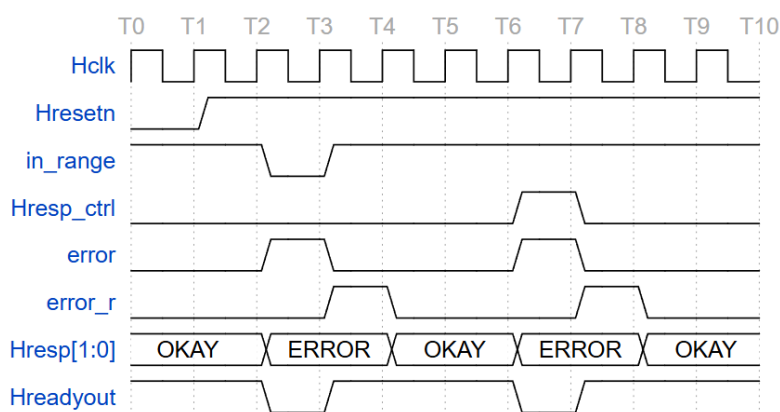
Khối phản hồi tín hiệu là phần xử lý cuối của khối giao diện tứ AHB, có nhiệm vụ tổng hợp thông tin từ máy trạng thái APB và khối giải mã địa chỉ. Từ đó, khối tạo ra các tín hiệu phản hồi phù hợp để gửi về AHB Master. Sơ đồ logic của khối phản hồi tín hiệu được thể hiện như hình 3.11.

**Hình 3.11:** Sơ đồ logic khối phản hồi tín hiệu

Hoạt động của khối phản hồi tín hiệu ở hình 3.11 như sau:

- Cơ chế phát hiện và ghi nhớ lỗi:
 - Khối phản hồi nhận lỗi từ hai nguồn: địa chỉ truy cập ngoài vùng hợp lệ và lỗi trả về từ phía APB thông qua tín hiệu Hresp_ctrl. Hai nguồn này được kết hợp để tạo ra tín hiệu error. Tín hiệu này được lưu qua thanh ghi error_r để giữ thông tin lỗi thêm một chu kỳ, phục vụ quá trình phản hồi lỗi về AHB Master.
- Điều khiển Hresp và Hreadyout:
 - Khi có lỗi, khối phản hồi tạo mã Hresp = ERROR trong hai chu kỳ liên tiếp. Ở chu kỳ đầu, Hreadyout được kéo xuống mức thấp để giữ Master chờ. Sang chu kỳ tiếp theo, Hreadyout được đưa lên mức cao để kết thúc giao dịch lỗi. Khi không có lỗi, Hreadyout được lấy theo tín hiệu Hreadyout_fsm từ máy trạng thái APB nhằm duy trì quá trình truyền nhận bình thường.

Cơ chế phản hồi lỗi hai chu kỳ được minh họa thông qua dạng sóng hoạt động của khối phản hồi tín hiệu ở hình 3.12.



Hình 3.12: Giảm đồ dạng sóng của phản hồi tín hiệu

Giảm đồ dạng sóng ở hình 3.12 minh họa về cơ chế phản hồi lỗi hai chu kỳ khi hệ thống phát hiện truy cập sai vùng địa chỉ:

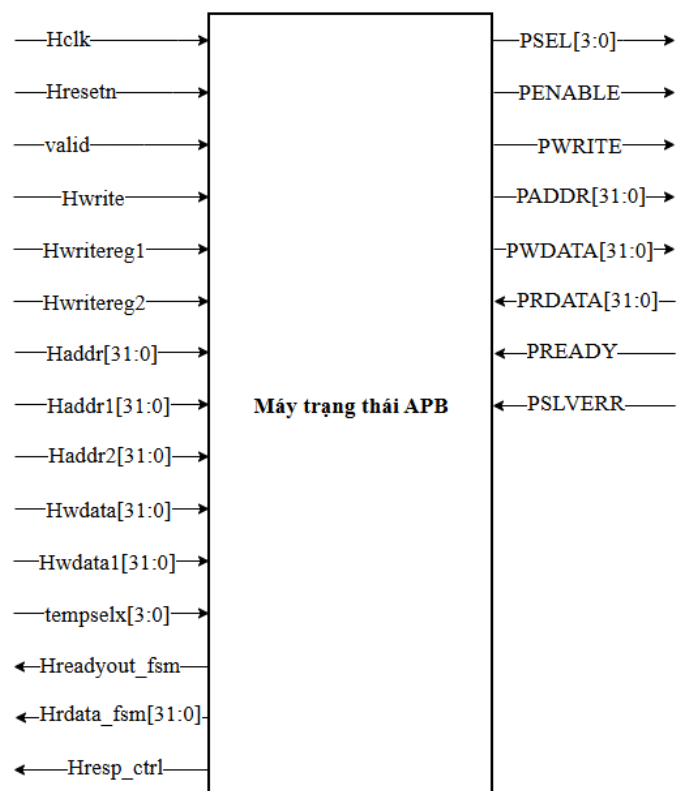
- Từ T2 đến T3, địa chỉ truy cập nằm ngoài vùng hợp lệ nên in_range ở mức thấp. Điều này làm tín hiệu error được kích hoạt, đồng thời Hreadyout bị kéo

xuống mức thấp để giữ Master chờ. Trong chu kỳ này, Hresp[1:0] chuyển từ OKAY sang ERROR, đánh dấu chu kỳ báo lỗi đầu tiên.

- Từ T3 đến T4, tín hiệu lỗi được lưu lại qua thanh ghi error_r. Nhờ đó, Hresp[1:0] tiếp tục giữ ở trạng thái ERROR thêm một chu kỳ, trong khi Hreadyout được đưa lên mức cao để Master có thể kết thúc giao dịch lỗi.
- Từ T6 đến T7, địa chỉ truy cập hợp lệ nhưng tín hiệu Hresp_ctrl từ phía điều khiển nội bộ được kích hoạt. Khi đó error cũng lên mức cao, làm Hreadyout bị kéo xuống thấp và Hresp[1:0] chuyển sang trạng thái ERROR.
- Từ T7 đến T8, lỗi được chốt lại trong error_r, giúp Hresp[1:0] tiếp tục duy trì trạng thái ERROR trong chu kỳ thứ hai. Đồng thời, Hreadyout trở lại mức cao để hoàn tất giao dịch. Sau T8, error_r trở về 0 và các tín hiệu phản hồi quay lại trạng thái bình thường.

3.3.2 Thiết kế máy trạng thái APB

Trong quá trình chuyển đổi từ AHB sang APB, máy trạng thái APB đảm nhiệm việc điều phối các bước truy cập đến ngoại vi. Các tín hiệu vào ra của khối này được trình bày ở hình 3.13.



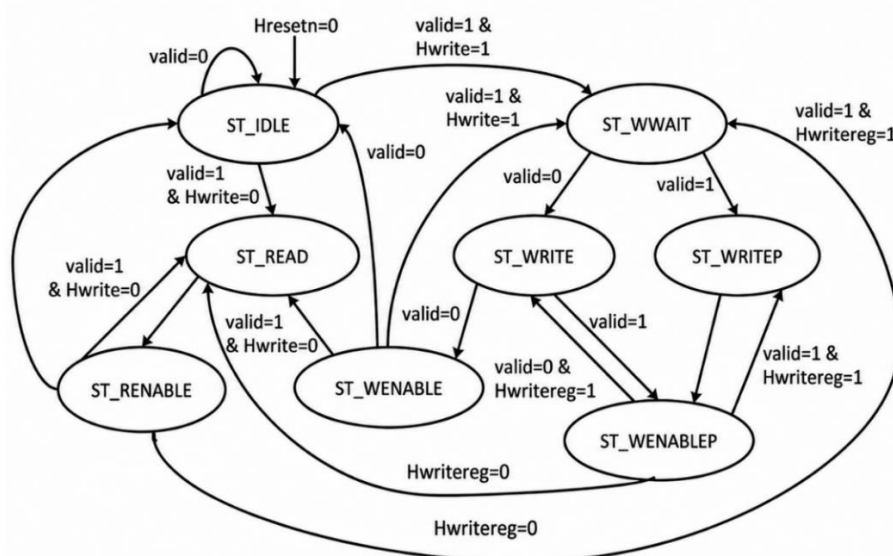
Hình 3.13: Các tín hiệu đầu vào và đầu ra của máy trạng thái APB

Máy trạng thái APB thực hiện việc điều khiển quá trình chuyển đổi giao dịch từ AHB sang APB. Khối này gồm tám trạng thái, tương ứng với các bước xử lý khác nhau của một giao dịch. Tên trạng thái và mã hóa tương ứng được trình bày trong bảng 3.4.

Bảng 3.4: Các trạng thái trong máy trạng thái APB

Trạng thái	Giá trị	Mô tả
ST_IDLE	3'b000	Trạng thái nghỉ.
ST_WWAIT	3'b001	Trạng thái chờ ghi.
ST_READ	3'b010	Trạng thái chuẩn bị đọc.
ST_WRITE	3'b011	Trạng thái chuẩn bị ghi đơn.
ST_WRITEP	3'b100	Trạng thái chuẩn bị ghi liên tiếp.
ST_RENABLE	3'b101	Trạng thái kích hoạt đọc.
ST_WENABLE	3'b110	Trạng thái kích hoạt ghi đơn.
ST_WENABLEP	3'b111	Trạng thái kích hoạt ghi liên tiếp.

Dựa trên thông tin giao dịch từ phía AHB, máy trạng thái APB tạo và điều khiển các tín hiệu cần thiết để thực hiện truy cập đến ngoại vi APB đúng trình tự. Sơ đồ chuyển trạng thái của khối được trình bày ở hình 3.14.



Hình 3.14: Sơ đồ máy trạng thái APB

Theo sơ đồ hình 3.14, máy trạng thái APB gồm tám trạng thái: ST_IDLE, ST_READ, ST_WWAIT, ST_WRITE, ST_WRITEP, ST_REENABLE, ST_WENABLE và ST_WENABLEP. Chức năng của từng trạng thái được mô tả như sau:

ST_IDLE: Trạng thái nghỉ

Bus APB đang ở chế độ chờ, không có giao dịch nào được thực hiện. Trạng thái này được kích hoạt khi hệ thống vừa reset hoặc sau khi hoàn tất một giao dịch mà không có yêu cầu mới.

Khi nhận được yêu cầu hợp lệ, máy trạng thái sẽ chuyển sang:

- ST_READ nếu Hwrite = 0 (yêu cầu đọc).
- ST_WWAIT nếu Hwrite = 1 (yêu cầu ghi).

ST_READ: Trạng thái chuẩn bị đọc

Máy trạng thái thực hiện pha SETUP của giao dịch đọc. Địa chỉ slave được đưa lên bus, các tín hiệu điều khiển được giữ ổn định trong khi tín hiệu PENABLE vẫn ở mức thấp, chờ bước kích hoạt tiếp theo. Trạng thái này được nhập từ ST_IDLE khi có yêu cầu đọc, hoặc từ ST_REENABLE khi có giao dịch đọc tiếp theo.

Sau khi thiết lập xong, máy trạng thái sẽ chuyển sang:

- ST_REENABLE để hoàn tất giao dịch đọc.

ST_REENABLE: Trạng thái kích hoạt đọc

Tín hiệu PENABLE được kích hoạt lên mức cao, slave nhận lệnh và đưa dữ liệu lên bus cho master đọc về. Đây là pha ACCESS và là bước cuối cùng của một giao dịch đọc hoàn chỉnh. Trạng thái này luôn được nhập từ ST_READ.

Sau khi hoàn tất giao dịch, máy trạng thái sẽ chuyển sang:

- ST_READ nếu có yêu cầu đọc tiếp theo.
- ST_WWAIT nếu có yêu cầu ghi tiếp theo.
- ST_IDLE nếu không còn giao dịch nào.

ST_WWAIT: Trạng thái chờ ghi

Sau khi nhận yêu cầu ghi từ master, máy trạng thái dừng lại ở đây để xác định kiểu giao dịch ghi sắp diễn ra. Nếu điều kiện chờ vẫn còn ($\text{valid}=1$, $\text{Hwritereg}=1$), máy trạng thái tự giữ nguyên tại đây. Trạng thái này luôn được nhập từ ST_IDLE.

Khi điều kiện chờ kết thúc, máy trạng thái sẽ chuyển sang:

- ST_WRITE nếu $\text{valid} = 0$, thực hiện giao dịch ghi đơn.
- ST_WRITEP nếu $\text{valid} = 1$, thực hiện giao dịch ghi pipeline.

ST_WRITE: Trạng thái chuẩn bị ghi đơn

Máy trạng thái thực hiện pha SETUP của giao dịch ghi đơn. Địa chỉ và dữ liệu cần ghi được đưa lên bus và giữ ổn định trong khi tín hiệu PENABLE vẫn ở mức thấp. Trạng thái này luôn được nhập từ ST_WWAIT khi $\text{valid}=0$.

Sau khi thiết lập xong, máy trạng thái sẽ chuyển sang:

- ST_WENABLE nếu $\text{Hwritereg} = 0$, không có giao dịch ghi nào đang chờ.
- ST_WENABLEP nếu $\text{Hwritereg} = 1$, có giao dịch ghi tiếp theo đang xếp hàng.

ST_WENABLE: Trạng thái kích hoạt ghi đơn

Tín hiệu PENABLE được kích hoạt lên mức cao, slave nhận dữ liệu và thực hiện ghi. Đây là pha ACCESS và là bước cuối cùng của một giao dịch ghi đơn hoàn chỉnh. Trạng thái này luôn được nhập từ ST_WRITE.

Sau khi hoàn tất giao dịch, máy trạng thái sẽ chuyển sang:

- ST_READ nếu có yêu cầu đọc tiếp theo.
- ST_WWAIT nếu có yêu cầu ghi tiếp theo.
- ST_IDLE nếu không còn giao dịch nào.

ST_WRITEP: Trạng thái chuẩn bị ghi liên tiếp

Máy trạng thái thực hiện pha SETUP của giao dịch ghi liên tiếp, xử lý trường hợp Master gửi yêu cầu ghi liên tiếp mà không chờ giao dịch trước hoàn tất. Dữ liệu và địa chỉ của giao dịch hiện tại được duy trì trên bus trong khi thông tin của

giao dịch tiếp theo đã được tiếp nhận. Trạng thái này luôn được nhập từ ST_WWAIT khi valid=1.

Sau khi thiết lập xong, máy trạng thái sẽ chuyển sang:

- ST_WENABLEP để hoàn tất giao dịch hiện tại.

ST_WENABLEP: Trạng thái kích hoạt ghi liên tiếp

Tín hiệu PENABLE được kích hoạt lên mức cao, Slave nhận và ghi dữ liệu tương tự ST_WENABLE. Điểm khác biệt là ngay tại thời điểm này, giao dịch ghi tiếp theo đã sẵn sàng để xử lý, giúp bus hoạt động liên tục mà không bị gián đoạn. Trạng thái này được nhập từ ST_WRITEP hoặc ST_WRITE.

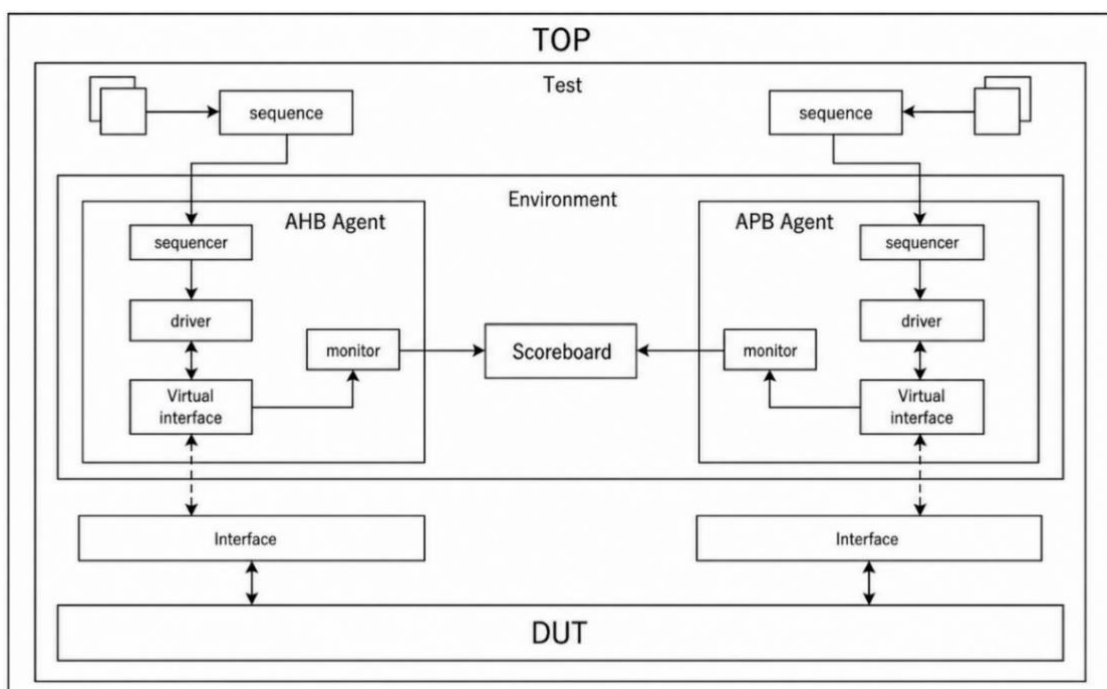
Sau khi hoàn tất giao dịch, máy trạng thái sẽ chuyển sang:

- ST_WRITEP nếu vẫn còn giao dịch ghi tiếp theo (valid=1, Hwritereg=1).
- ST_READ nếu không còn giao dịch nào và có giao dịch đọc kế tiếp (Hwritereg=0).
- ST_WRITE nếu vẫn còn giao dịch ghi cuối cùng (valid=0, Hwritereg=1).

3.4 THIẾT KẾ MÔI TRƯỜNG UVM

3.4.1 Tổng quan kiến trúc môi trường UVM

Môi trường kiểm thử UVM được xây dựng theo mô hình giao dịch, trong đó các thao tác kiểm thử được mô tả ở mức trừu tượng thay vì điều khiển trực tiếp từng tín hiệu. Cách tổ chức này giúp các kịch bản kiểm thử rõ ràng hơn, dễ mở rộng và tách biệt với phần thiết kế phần cứng. Sơ đồ tổng quát của môi trường UVM dùng để xác minh cầu chuyển đổi AHB – APB được trình bày ở hình 3.15.



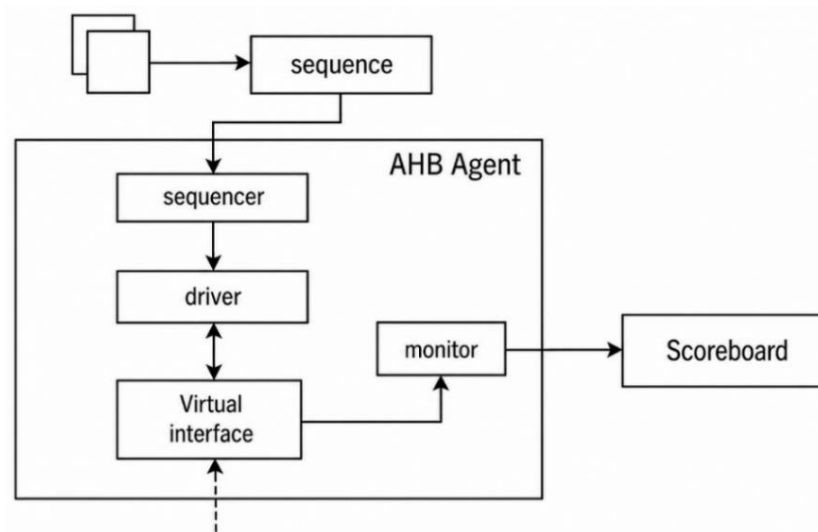
Hình 3.15: Sơ đồ môi trường UVM của cầu chuyển đổi AHB – APB

Dựa trên hình 3.15, môi trường xác minh UVM của cầu chuyển đổi AHB – APB gồm hai Agent chính. AHB Agent đóng vai trò chủ động, tạo các giao dịch AHB và đưa tín hiệu kích thích vào DUT. Trong khi đó, APB Agent thụ động theo dõi phản hồi từ phía APB và gửi dữ liệu thu thập được về Scoreboard để kiểm tra. Các kịch bản kiểm thử được xây dựng trong lớp UVM Test. Tùy theo từng kịch bản, Test sẽ kích hoạt các Sequence tương ứng thông qua Sequencer của từng Agent. Trong quá trình mô phỏng, các Monitor ở hai phía AHB và APB sẽ quan sát tín hiệu, đóng gói thành giao dịch và gửi về Scoreboard. Scoreboard sau đó thực hiện so sánh kết quả giữa dữ liệu mong đợi và dữ liệu thực tế.

Việc kết nối với DUT được thực hiện thông qua một giao diện dùng chung. Giao diện này chứa các tín hiệu vật lý và hỗ trợ đồng bộ theo xung nhịp, giúp quá trình truyền và lấy mẫu tín hiệu diễn ra đúng chu kỳ. Thay vì xử lý từng tín hiệu riêng lẻ, môi trường UVM đóng gói thông tin điều khiển và dữ liệu thành các đối tượng giao dịch, giúp quá trình kiểm thử rõ ràng và dễ quản lý hơn. Ở mức cao nhất, mô-đun Top chứa DUT và giao diện vật lý. Mô-đun này có nhiệm vụ tạo xung nhịp, reset hệ thống và đưa giao diện ảo vào cơ sở dữ liệu cấu hình để các thành phần UVM có thể sử dụng trong quá trình mô phỏng.

3.4.2 Thiết kế AHB Agent

AHB Agent giao tiếp với phía AHB của cầu chuyển đổi, tạo tín hiệu kiểm thử cho DUT và thu thập phản hồi để kiểm tra kết quả. Kiến trúc nội bộ của AHB Agent được trình bày ở hình 3.16.

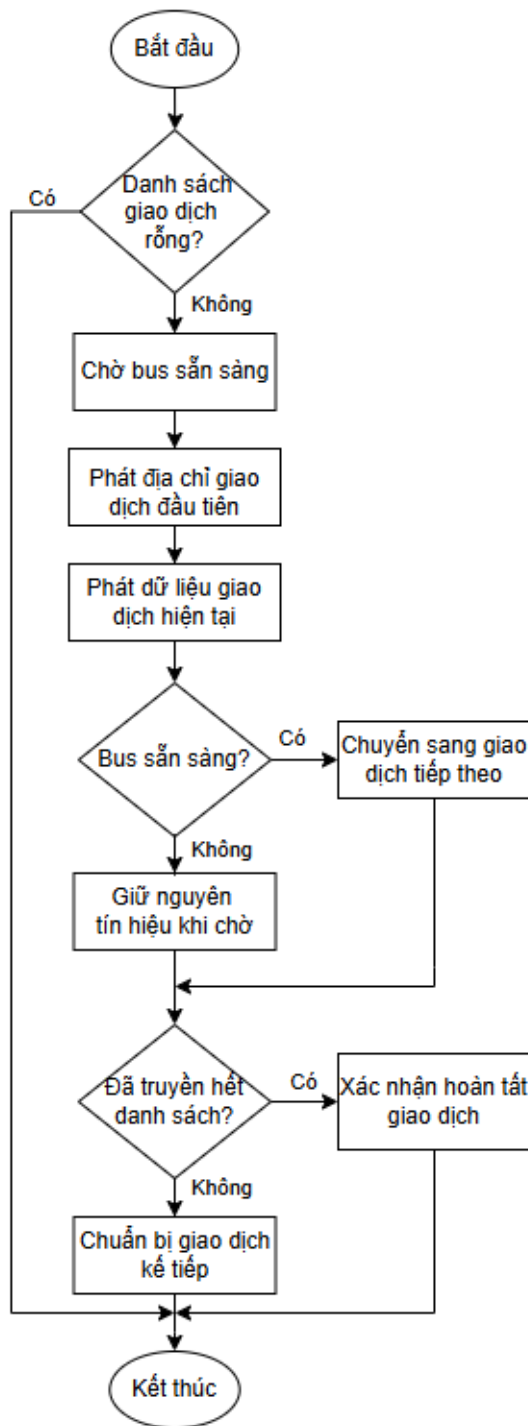


Hình 3.16: Kiến trúc nội bộ AHB Agent

Theo hình 3.16, AHB Agent gồm ba thành phần chính: Sequencer, Driver và Monitor. Sequencer quản lý các gói giao dịch và gửi lần lượt đến Driver. Driver nhận giao dịch từ Sequencer, sau đó chuyển thành tín hiệu AHB để đưa vào DUT thông qua virtual interface. Monitor theo dõi các tín hiệu trên bus AHB, đóng gói lại thành giao dịch và gửi đến Scoreboard để kiểm tra.

3.4.2.1 AHB Driver

AHB Driver có nhiệm vụ nhận giao dịch từ Sequencer và chuyển thành các tín hiệu điều khiển trên bus AHB. Việc điều khiển từng giao dịch được thực hiện trong một chương trình con riêng, như thể hiện ở hình 3.17.

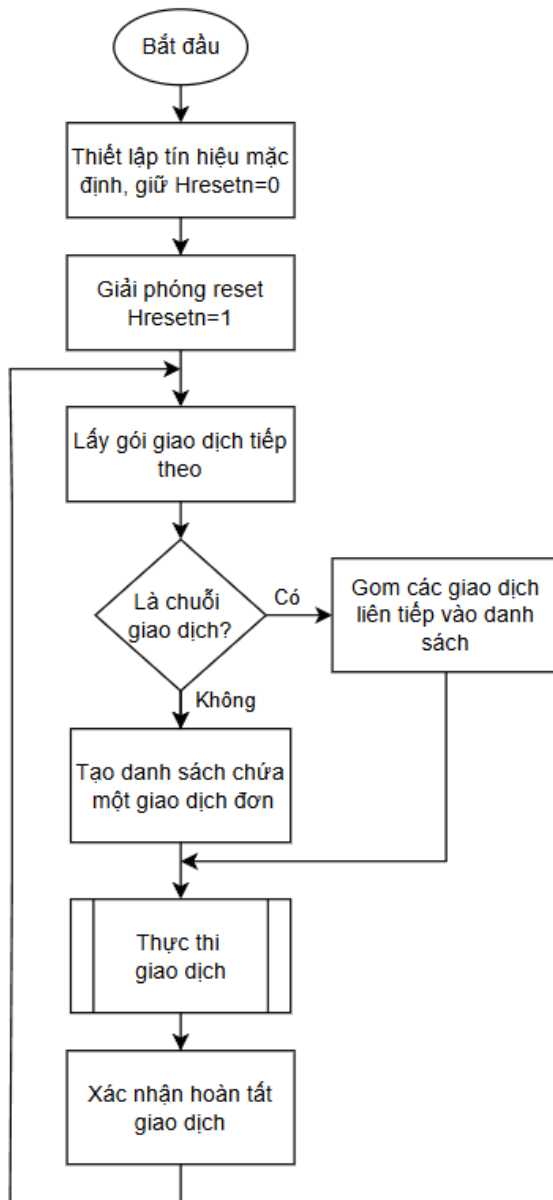


Hình 3.17: Lưu đồ chương trình con thực thi giao dịch của AHB Driver

Dựa trên lưu đồ hình 3.17, chương trình con thực thi giao dịch trước hết kiểm tra danh sách các giao dịch đang chờ truyền. Nếu danh sách không còn phần tử nào, chương trình con sẽ kết thúc và trả quyền điều khiển về chương trình chính. Khi vẫn còn giao dịch trong danh sách, Driver chờ bus AHB sẵn sàng trước khi bắt đầu truyền. Sau đó, Driver đưa địa chỉ và các tín hiệu điều khiển của giao dịch đầu tiên lên bus, đồng thời phát dữ liệu tương ứng cho giao dịch đang được xử lý. Trong lúc truyền, Driver liên tục theo dõi trạng thái sẵn sàng của bus. Nếu bus chưa sẵn sàng, các tín hiệu hiện tại sẽ được giữ nguyên để đảm bảo giao dịch không bị thay đổi trong chu kỳ chờ. Khi bus sẵn sàng, Driver hoàn tất giao dịch hiện tại và chuyển sang giao dịch kế tiếp.

Quá trình trên được lặp lại cho đến khi toàn bộ danh sách giao dịch đã được gửi đi. Sau khi hoàn tất, Driver gửi xác nhận hoàn thành giao dịch và chương trình con kết thúc.

Để có cái nhìn tổng quát về hoạt động của Driver, lưu đồ hoạt động được trình bày trong hình 3.18.



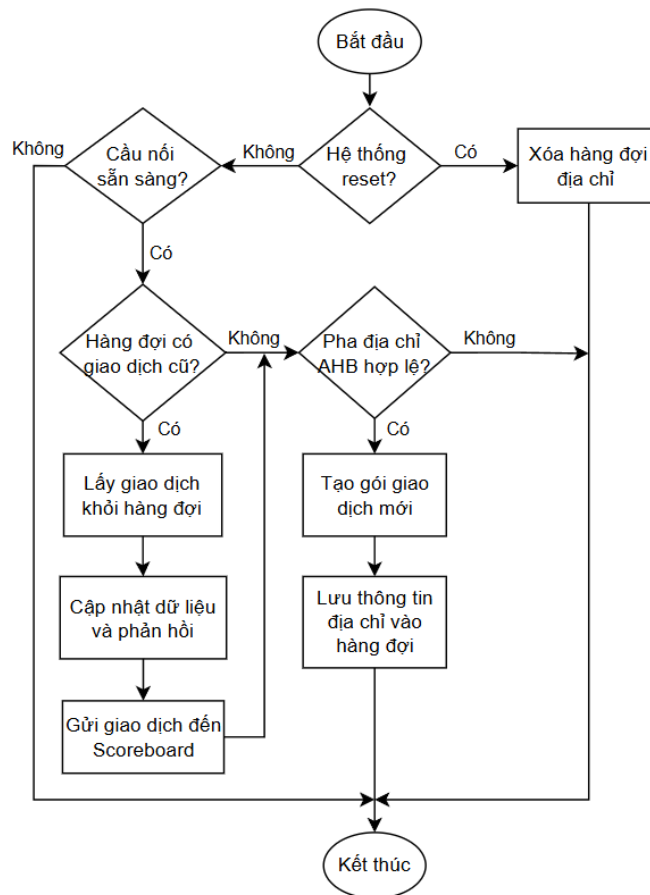
Hình 3.18: Lưu đồ hoạt động của AHB Driver

Ở hình 3.18, AHB Driver trước tiên khởi tạo các tín hiệu về giá trị mặc định và giữ Hresetn ở mức thấp để đưa bus về trạng thái ban đầu. Sau đó, Hresetn được đưa lên mức cao để kết thúc reset và cho phép Driver bắt đầu truyền giao dịch. Sau khi reset hoàn tất, Driver đi vào vòng lặp chính để nhận giao dịch từ Sequencer. Với mỗi giao dịch nhận được, Driver kiểm tra đó là giao dịch đơn hay một chuỗi giao dịch liên tiếp. Nếu là chuỗi, các giao dịch được đưa vào danh sách theo đúng thứ tự truyền. Nếu là giao dịch đơn, danh sách chỉ chứa giao dịch hiện tại.

Danh sách này sau đó được chuyển cho chương trình con thực thi giao dịch AHB. Chương trình con sẽ điều khiển các tín hiệu địa chỉ, dữ liệu và điều khiển trên bus theo trạng thái sẵn sàng của hệ thống. Khi truyền xong toàn bộ danh sách, Driver báo hoàn thành cho Sequencer và tiếp tục chờ giao dịch tiếp theo.

3.4.2.2 AHB Monitor

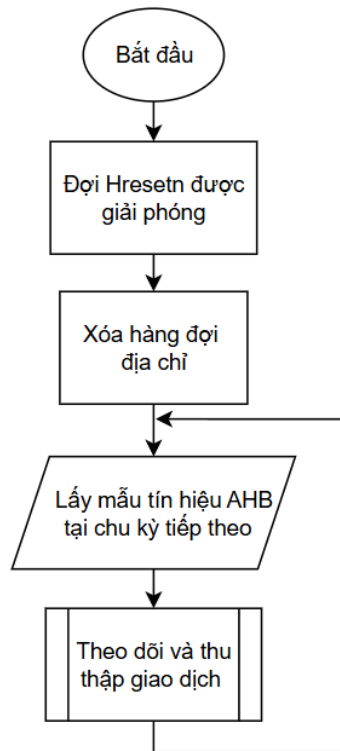
AHB Monitor hoạt động thụ động, có nhiệm vụ quan sát tín hiệu trên bus AHB, đóng gói thành giao dịch và gửi đến Scoreboard để kiểm tra. Việc phát hiện và ghi nhận từng giao dịch được thực hiện trong chương trình con theo dõi như ở hình 3.19.



Hình 3.19: Lưu đồ chương trình con theo dõi và thu thập giao dịch

Dựa trên hình 3.19, chương trình con theo dõi giao dịch AHB bắt đầu bằng việc kiểm tra tín hiệu reset. Nếu hệ thống đang reset, Monitor xóa hàng đợi địa chỉ để loại bỏ dữ liệu cũ, sau đó kết thúc chu kỳ theo dõi hiện tại. Khi hệ thống không reset, Monitor kiểm tra trạng thái sẵn sàng của cầu chuyển đổi. Nếu cầu chưa sẵn sàng, Monitor chưa thu thập giao dịch mới và kết thúc chu kỳ xử lý. Nếu cầu đã sẵn sàng, Monitor kiểm tra hàng đợi địa chỉ. Khi có giao dịch đang chờ hoàn tất, Monitor lấy giao dịch đó ra, bổ sung dữ liệu và tín hiệu phản hồi ở pha dữ liệu, sau đó gửi gói giao dịch hoàn chỉnh đến Scoreboard để kiểm tra. Sau đó, Monitor tiếp tục kiểm tra pha địa chỉ hiện tại. Nếu phát hiện một pha địa chỉ hợp lệ, Monitor tạo giao dịch mới và lưu thông tin địa chỉ vào hàng đợi để xử lý ở chu kỳ tiếp theo. Nếu không có pha địa chỉ hợp lệ, chương trình con kết thúc chu kỳ theo dõi.

Để có cái nhìn tổng quát về cách chương trình con này hoạt động trong AHB Monitor, lưu đồ hoạt động chính được trình bày trong hình 3.20.



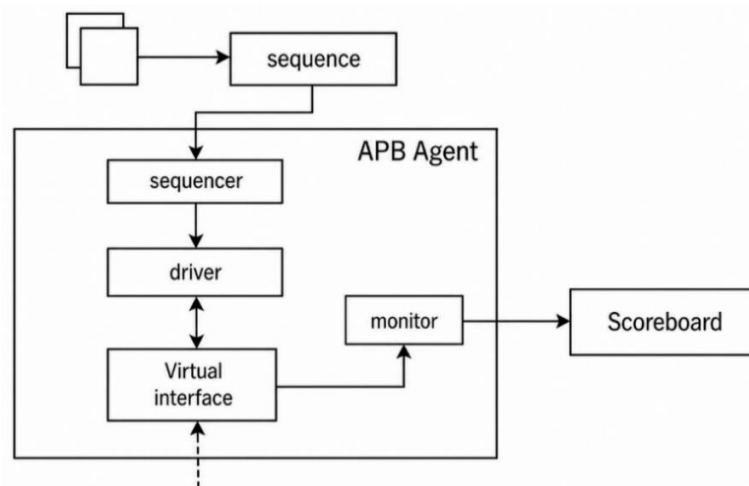
Hình 3.20 trình bày quá trình hoạt động chính của AHB Monitor. Sau khi hệ thống thoát khỏi reset, Monitor xóa hàng đợi địa chỉ để bắt đầu trạng thái theo dõi mới.

Trong quá trình mô phỏng, Monitor lấy mẫu các tín hiệu AHB theo từng chu kỳ xung nhịp và gọi chương trình con thu thập giao dịch. Nhờ đó, Monitor có thể theo dõi liên tục hoạt động trên bus, ghi nhận các giao dịch hợp lệ và gửi thông tin đã đóng gói đến Scoreboard để kiểm tra.

Hình 3.20: Lưu đồ hoạt động của AHB Monitor

3.4.3 Thiết kế APB Agent

APB Agent đóng vai trò mô phỏng các ngoại vi APB để nhận và phản hồi giao dịch từ DUT. Kiến trúc của APB Agent được trình bày ở hình 3.21.

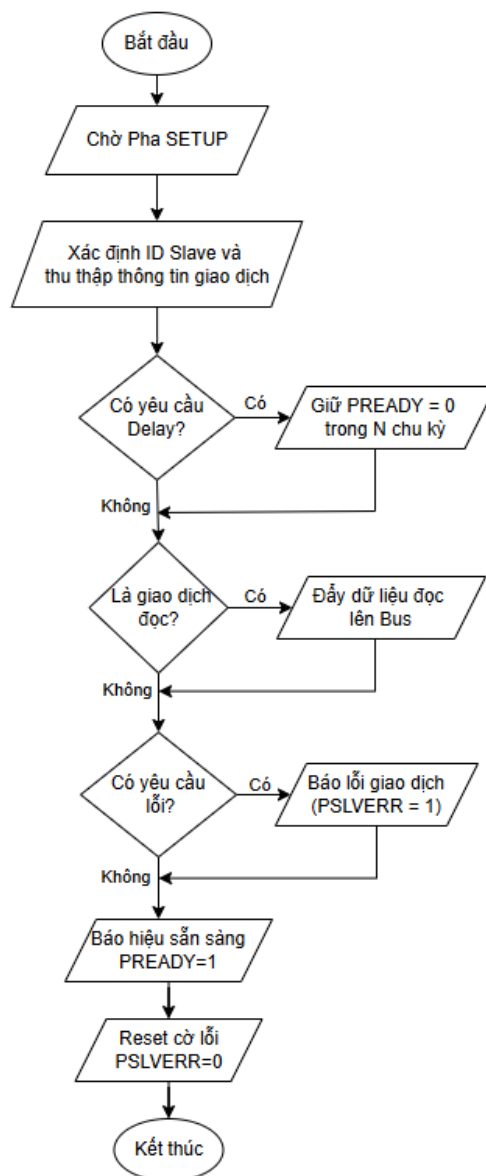


Hình 3.21: Kiến trúc nội bộ APB Agent

Hình 3.21 cho thấy APB Agent có cấu trúc gồm Driver và Monitor. APB Driver nhận tín hiệu từ DUT và tạo phản hồi tương ứng như một APB Slave. APB Monitor theo dõi các giao dịch trên bus APB, đóng gói thông tin thu được và gửi đến Scoreboard để đối chiếu với phía AHB.

3.4.3.1 APB Driver

Khác với AHB Driver, APB Driver không tự tạo giao dịch mà chỉ theo dõi các tín hiệu do DUT phát ra và đưa phản hồi phù hợp. Quá trình này được thực hiện trong chương trình con phản hồi giao dịch APB, có nhiệm vụ phát hiện giao dịch và tạo tín hiệu đáp ứng cho từng lần truy cập. Lưu đồ của chương trình con được trình bày ở hình 3.22.



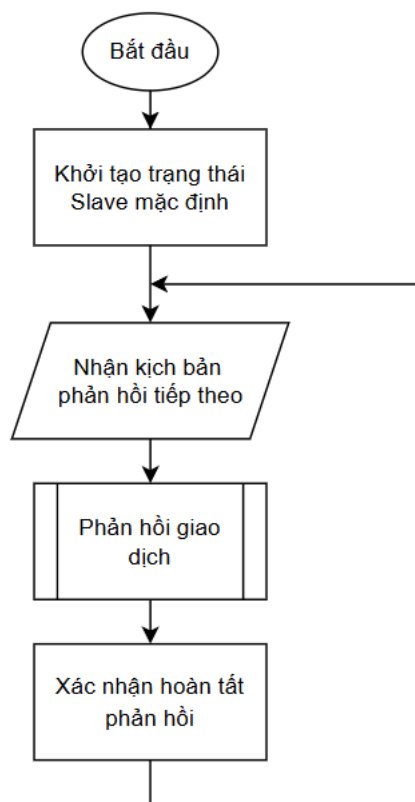
Hình 3.22: Lưu đồ chương trình con phản hồi giao dịch

Dựa trên hình 3.22, APB Driver bắt đầu bằng việc theo dõi interface và chờ DUT đưa giao dịch vào pha SETUP. Khi pha SETUP xuất hiện, Driver xác định Slave được chọn, đồng thời ghi nhận địa chỉ và loại giao dịch đọc hoặc ghi.

Sau đó, Driver kiểm tra có cần tạo độ trễ phản hồi hay không. Nếu có, tín hiệu PREADY được giữ ở mức 0 trong số chu kỳ đã đặt. Nếu không có độ trễ, Driver tiếp tục xử lý giao dịch. Với giao dịch đọc, dữ liệu được đưa lên bus để trả về DUT.

Tiếp theo, Driver kiểm tra kích bản lỗi. Nếu lỗi được kích hoạt, tín hiệu PSLVERR được đưa lên mức 1 để kiểm tra khả năng truyền lỗi từ APB về AHB. Cuối cùng, PREADY được kéo lên mức 1 để hoàn tất giao dịch, sau đó PSLVERR được xóa về 0 để không ảnh hưởng đến giao dịch tiếp theo.

Để có cái nhìn tổng quát về hoạt động của APB Driver, lưu đồ hoạt động chính được thể hiện trong hình 3.23.



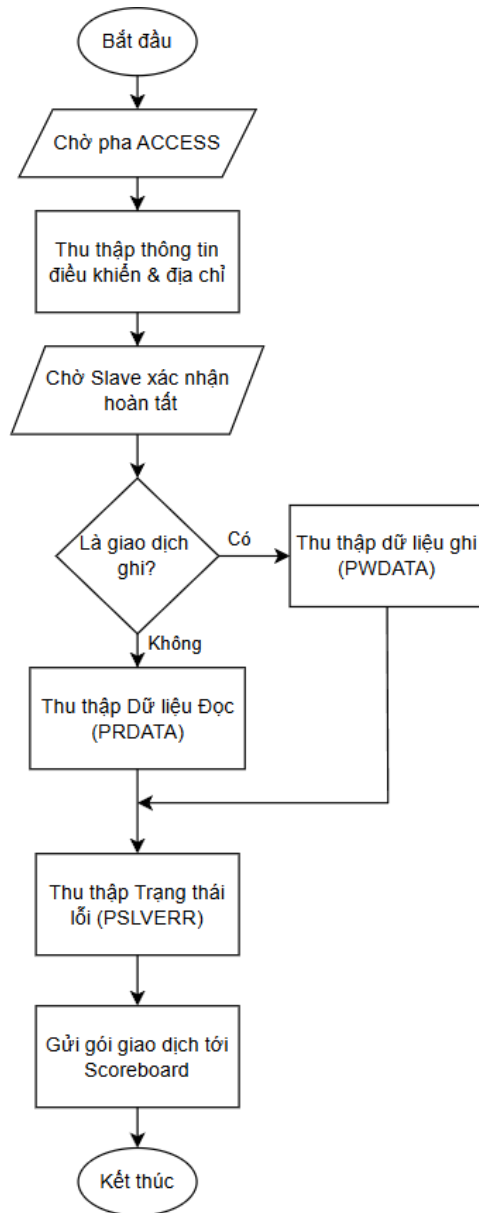
Hình 3.23: Lưu đồ hoạt động của APB Driver

Ở hình 3.23, APB Driver bắt đầu bằng việc đưa các tín hiệu phản hồi của Slave về trạng thái mặc định, như PREADY và PSLVERR, để đảm bảo hệ thống ổn định trước khi mô phỏng.

Sau đó, Driver đi vào vòng lặp chính. Trong mỗi vòng lặp, Driver nhận kích bản phản hồi từ Sequencer, gọi chương trình con xử lý giao dịch APB và gửi xác nhận hoàn tất sau khi phản hồi xong. Quá trình này được lặp lại liên tục, giúp APB Slave luôn sẵn sàng đáp ứng các giao dịch do DUT phát ra trong suốt quá trình mô phỏng.

3.4.3.2 APB Monitor

APB Monitor có nhiệm vụ quan sát các tín hiệu trên bus APB, đóng gói thành giao dịch và gửi đến Scoreboard để kiểm tra với dữ liệu phía AHB. Việc phát hiện và ghi nhận từng giao dịch được thực hiện trong chương trình con theo dõi, như trình bày ở hình 3.24.



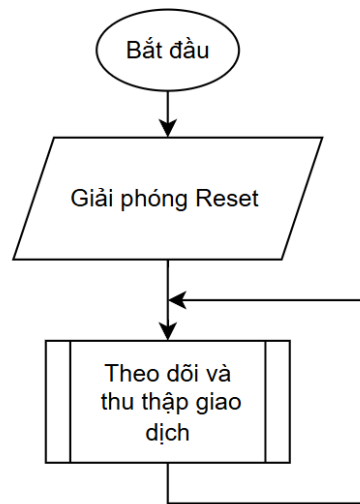
Hình 3.24: Lưu đồ chương trình con theo dõi và thu thập giao dịch

Dựa trên hình 3.24, APB Monitor chờ đến pha ACCESS để xác nhận giao dịch APB đang được thực hiện. Khi điều kiện này xuất hiện, Monitor thu thập các thông tin như địa chỉ, tín hiệu điều khiển và loại giao dịch.

Sau đó, Monitor tiếp tục theo dõi PREADY cho đến khi Slave báo hoàn tất. Nếu là giao dịch ghi, Monitor lấy dữ liệu từ PDATA; nếu là giao dịch đọc, Monitor lấy dữ liệu từ PRDATA. Đồng thời, trạng thái lỗi cũng được ghi nhận thông qua tín hiệu PSLVERR.

Cuối cùng, các thông tin thu được được đóng gói thành một giao dịch APB và gửi đến Scoreboard để phục vụ quá trình đối chiếu kết quả.

Để có cái nhìn tổng quát về cách hoạt động của APB Monitor, lưu đồ hoạt động của APB Monitor được trình bày trong hình 3.25.



Hình 3.25: Lưu đồ hoạt động của APB Monitor

Ở hình 3.25, APB Monitor bắt đầu bằng việc chờ hệ thống thoát khỏi reset để đảm bảo các tín hiệu đã ổn định. Sau đó, Monitor đi vào vòng lặp chính và liên tục gọi chương trình con theo dõi giao dịch. Quá trình này được lặp lại trong suốt thời gian mô phỏng, giúp Monitor luôn quan sát bus APB và thu thập các giao dịch phát sinh để gửi đến Scoreboard.

3.4.4 Thiết kế Scoreboard

Scoreboard nhận dữ liệu từ hai nguồn: AHB Monitor và APB Monitor. Do thời điểm giao dịch xuất hiện ở hai bus có thể khác nhau, Scoreboard sẽ lưu tạm gói dữ liệu đến trước và chờ gói tương ứng từ phía còn lại. Khi cả hai gói của cùng một giao dịch đã có đủ, quá trình so sánh mới được thực hiện.

Khi nhận giao dịch từ AHB Monitor, Scoreboard trước hết kiểm tra địa chỉ. Nếu địa chỉ nằm ngoài vùng APB hợp lệ từ 0x4000_0000 đến 0x4000_3FFF, Scoreboard chỉ kiểm tra phản hồi lỗi từ DUT về phía AHB, vì giao dịch này không được chuyển xuống bus APB.

Với các giao dịch có địa chỉ hợp lệ, dữ liệu từ phía AHB được lưu vào hàng đợi nội bộ. Khi APB Monitor gửi gói giao dịch tương ứng, Scoreboard sẽ đối chiếu hai phía theo các tiêu chí kiểm tra được trình bày trong bảng 3.5.

Bảng 3.5: Các điều kiện kiểm thử chức năng của Scoreboard

STT	Điều kiện kiểm tra	Ý nghĩa
1	$PADDR == Haddr$	Địa chỉ được chuyển chính xác từ AHB sang APB
2	$PWDATA == Hwdata$	Dữ liệu ghi được truyền nguyên vẹn
3	$Hrdata == PRDATA$	Dữ liệu đọc trả về đúng chiều về AHB Master
4	$PSLVERR = 1 \rightarrow Hresp = ERROR$	Lỗi từ Slave được báo ngược lên AHB Master

Sau mỗi lần so sánh, Scoreboard ghi nhận kết quả kiểm tra. Nếu dữ liệu hai phía khớp nhau, giao dịch được đánh dấu thành công; nếu có sai lệch, Scoreboard sẽ phát cảnh báo lỗi để phục vụ quá trình phân tích.

Chương 4

KẾT QUẢ

4.1 KẾT QUẢ MÔ PHỎNG CHỨC NĂNG

4.1.1 Các trường hợp kiểm thử chức năng của hệ thống

Đối với một thiết kế cầu chuyển đổi giữa hai giao thức có đặc tính truyền dữ liệu và tốc độ phản hồi khác nhau như AHB – APB, việc tách biệt các trường hợp kiểm thử riêng biệt là cần thiết. Điều này cho phép cô lập từng hành vi của giao thức, từ các giao dịch đơn lẻ đến các chuỗi truyền dữ liệu liên tục.

Nhằm đánh giá chức năng của cầu chuyển đổi AHB – APB, các kịch bản kiểm thử đặc trưng dùng để kiểm tra các trạng thái hoạt động của hệ thống được tóm tắt trong bảng 4.1.

Bảng 4.1: Tóm tắt các kịch bản kiểm thử chức năng hệ thống

STT	Tên kịch bản	Mục đích
1	Giao dịch đọc và ghi đơn xen kẽ ngẫu nhiên	Xác minh chức năng điều hướng giao dịch cơ bản của hệ thống.
2	Chuỗi giao dịch ghi liên tiếp	Đánh giá khả năng chuyển đổi chuỗi dữ liệu gói đầu liên tục.
3	Chuỗi giao dịch đọc liên tiếp	Xác minh tính ổn định của đường truyền dữ liệu phản hồi từ APB Slave về AHB Master.
4	Phản hồi lỗi về Master	Kiểm tra cơ chế phản hồi khi ngoại vi kích hoạt tín hiệu báo lỗi.

Dựa trên các kịch bản trong bảng 4.1, thiết kế được kiểm tra theo từng nhóm chức năng, bắt đầu từ các giao dịch cơ bản đến các trường hợp phản hồi lỗi. Các thông số đầu vào dùng để triển khai từng kịch bản kiểm thử được trình bày trong bảng 4.2.

Bảng 4.2: Các kịch bản kiểm thử cho cầu chuyển đổi AHB – APB

Testcase	Mô tả	Cấu hình
Testcase 1	Cầu chuyển đổi AHB – APB ở quá trình ghi và đọc xen kẽ	Giai đoạn 1: Khởi tạo hệ thống Kích hoạt reset để đưa toàn bộ tín hiệu điều khiển về trạng thái ban đầu Giai đoạn 2: Thực hiện giao dịch ghi đơn Gửi tín hiệu thông báo loại giao dịch: <i>HTRANS = NONSEQ</i> Gửi tín hiệu thông báo yêu cầu ghi: <i>HWRITE = 1</i> Gửi tín hiệu địa chỉ vùng ngoại vi APB cần ghi: <i>HADDR = [Địa chỉ cần ghi]</i> Gửi tín hiệu chứa dữ liệu ghi: <i>HWDATA = [Dữ liệu ghi]</i> Giai đoạn 3: Thực hiện giao dịch đọc đơn Gửi tín hiệu thông báo loại giao dịch: <i>HTRANS = NONSEQ</i> Gửi tín hiệu thông báo yêu cầu đọc: <i>HWRITE = 0</i> Gửi tín hiệu địa chỉ vùng ngoại vi APB cần đọc: <i>HADDR = [Địa chỉ cần đọc]</i>
Testcase 2	Cầu chuyển đổi AHB – APB ở quá trình ghi liên tiếp	Giai đoạn 1: Khởi tạo hệ thống Kích hoạt reset để đưa toàn bộ tín hiệu điều khiển về trạng thái ban đầu. Giai đoạn 2: Truyền nhịp đầu tiên của quá trình ghi liên tiếp:

		Gửi tín hiệu thông báo loại giao dịch đầu tiên: <i>HTRANS = NONSEQ</i> Gửi tín hiệu thông báo yêu cầu ghi: <i>HWRITE = 1</i> Gửi tín hiệu địa chỉ bắt đầu của burst: <i>HADDR = [Địa chỉ bắt đầu của burst]</i> Gửi tín hiệu chứa dữ liệu ghi đầu tiên: <i>HWDATA = [Dữ liệu ghi đầu tiên]</i> Giai đoạn 3: Truyền các nhịp kế tiếp: Gửi tín hiệu thông báo loại giao dịch tuần tự cho các nhịp sau: <i>HTRANS = SEQ</i> Gửi tín hiệu duy trì yêu cầu ghi: <i>HWRITE = 1</i> Cập nhật tín hiệu địa chỉ: <i>HADDR = [Địa chỉ thay đổi theo kiểu burst]</i> Cập nhật tín hiệu dữ liệu: <i>HWDATA = [Dữ liệu thay đổi theo nhịp]</i>
Testcase 3	Cầu chuyển đổi AHB – APB ở quá trình đọc liên tiếp	Giai đoạn 1: Khởi tạo hệ thống Kích hoạt reset để đưa toàn bộ tín hiệu điều khiển về trạng thái ban đầu. Giai đoạn 2: Khởi tạo nhịp đầu tiên của burst đọc Gửi tín hiệu thông báo loại giao dịch đầu tiên: <i>HTRANS = NONSEQ</i> Gửi tín hiệu thông báo yêu cầu đọc:

		$HWRITE = 0$ Gửi tín hiệu địa chỉ bắt đầu của burst đọc: $HADDR = [Địa\ chỉ\ bắt\ đầu\ của\ burst]$ Giai đoạn 3: Truyền các nhịp đọc kế tiếp: Gửi tín hiệu thông báo loại giao dịch tuần tự cho các nhịp sau: $HTRANS = SEQ$ Gửi tín hiệu duy trì yêu cầu đọc: $HWRITE = 0$ Cập nhật tín hiệu địa chỉ (tăng tuần tự theo kích thước truyền): $HADDR = [Địa\ chỉ\ thay\ đổi\ theo\ kiểu\ burst]$
Testcase 4	Cầu chuyển đổi AHB – APB ở quá trình kiểm tra cơ chế phản hồi lỗi từ ngoại vi APB	Giai đoạn 1: Thiết lập giao dịch truy cập ngoại vi: Gửi tín hiệu thông báo loại giao dịch hợp lệ: $HTRANS = NONSEQ$ Gửi tín hiệu thông báo yêu cầu đọc hoặc ghi: $HWRITE = 1\ hoặc\ 0$ Gửi tín hiệu địa chỉ ngoại vi cần truy cập: $HADDR = [Địa\ chỉ\ ngoại\ vi]$ Giai đoạn 2: Mô phỏng phản hồi từ ngoại vi: Xác nhận hệ thống đang chuyển sang pha ACCESS của chuẩn APB: $PSELx = 1, PENABLE = 1$ Kích hoạt tín hiệu báo lỗi ngoại vi: $PSLVERR = 1$

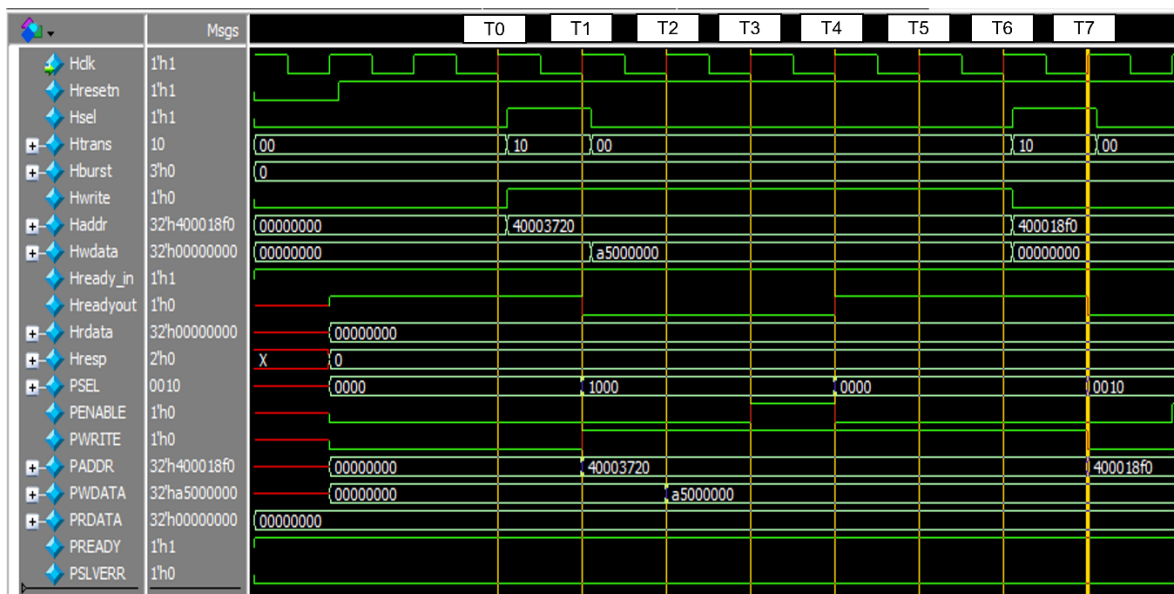
4.1.2 Kết quả mô phỏng

4.1.2.1 Testcase 1: Giao dịch ghi đơn và đọc đơn xen kẽ

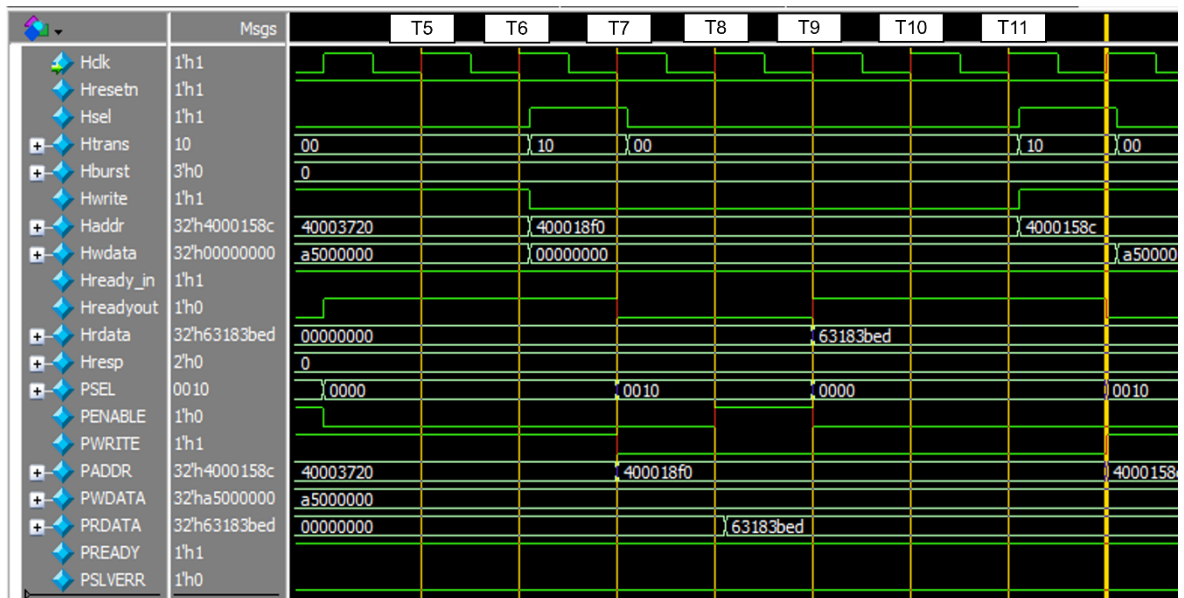
Testcase 1 kiểm tra hoạt động cơ bản của cầu chuyển đổi thông qua các giao dịch ghi và đọc đơn lẻ được thực hiện luân phiên. Mỗi giao dịch là một lần truyền độc lập, không thuộc chuỗi burst.

Hình 4.1 trình bày kết quả của giao dịch ghi đơn từ T0 đến T5. Trong quá trình này, địa chỉ PADDR và dữ liệu ghi PWDATA được chuyển từ phía AHB sang APB qua hai pha SETUP và ACCESS.

Hình 4.2 mô tả kết quả giao dịch đọc đơn tiếp theo từ T5 đến T11. Cầu chuyển đổi thực hiện truy cập đọc trên bus APB, sau đó đưa dữ liệu đọc về phía AHB Master thông qua tín hiệu HRDATA.



Hình 4.1: Mô phỏng cầu chuyển đổi AHB – APB ở kịch bản giao dịch ghi đơn



Hình 4.2: Mô phỏng cầu chuyển đổi AHB – APB ở kịch bản giao dịch đọc đơn

Kết quả mô phỏng tại hình 4.1 và hình 4.2 được phân tích theo từng giai đoạn như sau:

- Từ T0 đến T1:
 - AHB Master bắt đầu một giao dịch ghi đơn. Các tín hiệu được thiết lập gồm Hsel = 1, Htrans = 2'b10 tương ứng NONSEQ, Hburst = 3'b000 tương ứng SINGLE, Hwrite = 1 và Haddr = 32'h40003720. Địa chỉ này thuộc vùng của Slave 3 nên khối giải mã kích hoạt tín hiệu chọn tương ứng PSEL = 4'b1000.
- Từ T1 đến T2:
 - Sau khi nhận giao dịch ghi hợp lệ, máy trạng thái chuyển từ ST_IDLE sang ST_WWAIT. Địa chỉ Haddr = 32'h40003720 được lưu vào khối đường ống. Ở phía APB, các tín hiệu bắt đầu được thiết lập với PSEL = 4'b1000, PWRITE = 1, PADDR = 32'h40003720 và PENABLE = 0.
- Từ T2 đến T3:
 - Dữ liệu ghi Hwdata = 32'ha5000000 xuất hiện ở pha dữ liệu của AHB và được chuyển sang APB qua tín hiệu PWDATA = 32'ha5000000. Máy trạng thái ở ST_WRITE, tương ứng pha chuẩn bị ghi của APB.

Các tín hiệu PSEL, PWRITE, PADDR và PWDATA đã ổn định, trong khi Hreadyout vẫn ở mức thấp để giữ AHB Master chờ.

- Từ T3 đến T4:
 - PENABLE được đưa lên mức cao, chuyển giao dịch APB sang pha truy cập. Khi đó, các tín hiệu APB gồm $PSEL = 4'b1000$, $PWRITE = 1$, $PADDR = 32'h40003720$, $PWDATA = 32'ha5000000$ và $PENABLE = 1$. Do $PREADY = 1$ và $PSLVERR = 0$, giao dịch ghi hoàn tất thành công. DUT phản hồi $HRESP = 2'b00$ tương ứng OKAY và đưa Hreadyout lên mức cao để nhận giao dịch tiếp theo.
- Từ T4 đến T5:
 - Sau khi giao dịch ghi kết thúc, máy trạng thái quay về ST_IDLE. Tín hiệu PSEL trở về $4'b0000$ và $PENABLE = 0$, cho thấy không có APB Slave nào đang được truy cập.
- Từ T5 đến T6:
 - Hệ thống ở trạng thái rỗi sau giao dịch ghi. Hreadyout tiếp tục giữ mức cao, cho biết cầu chuyển đổi đã sẵn sàng nhận yêu cầu mới. Cuối khoảng này, AHB Master bắt đầu phát giao dịch đọc đơn tiếp theo.
- Từ T6 đến T7:
 - AHB Master phát giao dịch đọc đơn với $Hsel = 1$, $Htrans = 2'b10$, $Hburst = 3'b000$, $Hwrite = 0$ và $Haddr = 32'h400018f0$. Đây là giao dịch đọc hợp lệ nên máy trạng thái chuẩn bị chuyển từ ST_IDLE sang ST_READ.
- Từ T7 đến T8:
 - Máy trạng thái chuyển sang ST_READ, tương ứng pha chuẩn bị đọc của APB. Các tín hiệu APB được thiết lập gồm $PSEL = 4'b0010$, $PWRITE = 0$, $PADDR = 32'h400018f0$ và $PENABLE = 0$. Trong thời gian này, Hreadyout được kéo xuống mức thấp để báo AHB Master chờ trong khi giao dịch đọc đang được xử lý.

- Từ T8 đến T9:
 - Máy trạng thái chuyển sang ST_REENABLE. Tín hiệu PENABLE được đưa lên mức cao, còn PSEL, PWRITE và PADDR vẫn giữ ổn định. APB Slave trả dữ liệu đọc qua PRDATA = 32'h63183bed. Vì PREADY = 1 và PSLVERR = 0, giao dịch đọc hoàn tất thành công và dữ liệu được cầu chuyển đổi nhận về để trả lại phía AHB.
- Từ T9 đến T10:
 - Dữ liệu đọc được đưa lên HRDATA = 32'h63183bed. Đồng thời, HRESP = 2'b00 cho biết phản hồi OKAY và Hreadyout trở lại mức cao, xác nhận giao dịch đọc đã hoàn tất.
- Từ T10 đến T11:
 - Hệ thống tiếp tục ở trạng thái rồi do phía AHB chưa phát giao dịch hợp lệ mới, thể hiện qua Htrans = 2'b00. Hreadyout vẫn giữ mức cao. Gần thời điểm T11, Master bắt đầu phát giao dịch mới với Htrans = 2'b10 và Haddr = 32'h4000158c; máy trạng thái sẽ tiếp tục kiểm tra loại giao dịch để xử lý bước tiếp theo.

Để xác nhận tính đúng đắn của toàn bộ kịch bản, hình 4.3 trình bày kết quả kiểm tra tự động của Scoreboard sau khi hoàn tất 10 giao dịch xen kẽ ghi và đọc đơn.

TYPE	PKT	BURST	TRANS	DATA (AHB)	DATA (APB)	HADDR	PADDR	SLVERR	HRESP	STATUS
WRITE	[0]	SINGLE	NONSEQ	a5000000	a5000000	40003720	40003720	0	OKAY	PASS
READ	[1]	SINGLE	NONSEQ	63183bed	63183bed	400018f0	400018f0	0	OKAY	PASS
WRITE	[2]	SINGLE	NONSEQ	a5000002	a5000002	4000158c	4000158c	0	OKAY	PASS
READ	[3]	SINGLE	NONSEQ	85214b11	85214b11	40002ec0	40002ec0	0	OKAY	PASS
WRITE	[4]	SINGLE	NONSEQ	a5000004	a5000004	4000207c	4000207c	0	OKAY	PASS
READ	[5]	SINGLE	NONSEQ	788fa446	788fa446	400037c0	400037c0	0	OKAY	PASS
WRITE	[6]	SINGLE	NONSEQ	a5000006	a5000006	40000a20	40000a20	0	OKAY	PASS
READ	[7]	SINGLE	NONSEQ	7a240f6e	7a240f6e	40002090	40002090	0	OKAY	PASS
WRITE	[8]	SINGLE	NONSEQ	a5000008	a5000008	40000294	40000294	0	OKAY	PASS
READ	[9]	SINGLE	NONSEQ	476f9f7a	476f9f7a	4000126c	4000126c	0	OKAY	PASS

UVM_INFO testbench/env/bridge_scoreboard.sv(196) @ 775: uvm_test_top.env.sb [bridge_scoreboard]
 SCOREBOARD TONG KET: PASS = 10 | FAIL = 0

Hình 4.3: Kết quả kiểm tra của Scoreboard ở testcase 1

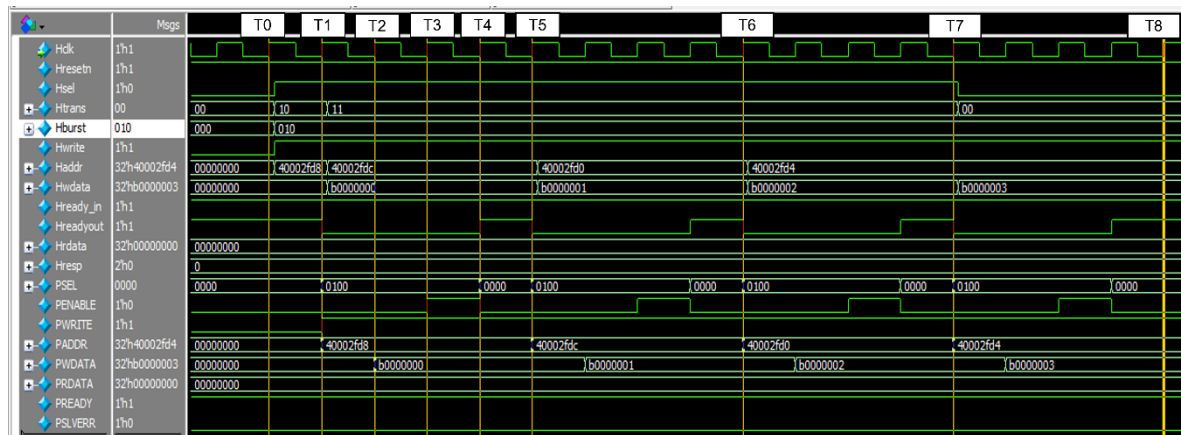
Kết quả mô phỏng cho thấy 10 giao dịch đều được kiểm tra thành công, với HRESP = OKAY và SLVERR = 0. Dữ liệu giữa phía AHB và APB khớp nhau, đồng thời địa chỉ HADDR được chuyển tiếp đúng sang PADDR. Scoreboard ghi nhận

PASS = 10 và FAIL = 0, cho thấy cầu chuyển đổi hoạt động đúng với các giao dịch ghi đơn và đọc đơn cơ bản.

4.1.2.2 Testcase 2: Chuỗi giao dịch ghi liên tiếp

Testcase 2 kiểm tra khả năng xử lý các giao dịch ghi liên tiếp khi AHB Master phát burst với nhiều kiểu truyền khác nhau. Trọng tâm của testcase là đánh giá cách cầu chuyển đổi duy trì và xử lý các giao dịch gói đầu giữa AHB và APB.

Hình 4.4 trình bày dạng sóng của kịch bản này, thể hiện sự thay đổi của các tín hiệu trên hai bus AHB và APB trong quá trình thực hiện chuỗi ghi liên tiếp.



Hình 4.4: Mô phỏng cầu AHB – APB ở kịch bản chuỗi giao dịch ghi liên tiếp

Kết quả mô phỏng tại hình 4.4 được phân tích theo từng giai đoạn như sau:

- Từ T0 đến T1:
 - AHB Master bắt đầu nhịp đầu tiên của burst ghi với Htrans = 2'b10 tương ứng NONSEQ, Hburst = 3'b010 tương ứng WRAP4, Hwrite = 1 và Haddr = 32'h40002fd8. Đây là pha địa chỉ của nhịp ghi đầu tiên. Địa chỉ này thuộc vùng của Slave 2 nên tín hiệu chọn phía APB là PSEL = 4'b0100. Khi phát hiện giao dịch hợp lệ, máy trạng thái chuyển từ ST_IDLE sang ST_WWAIT để chờ dữ liệu ghi ở chu kỳ tiếp theo.
- Từ T1 đến T2:
 - Master tiếp tục phát nhịp thứ hai của burst với Htrans = 2'b11 tương ứng SEQ và Haddr = 32'h40002fdc. Cùng lúc đó, dữ liệu của nhịp ghi đầu tiên xuất hiện trên bus AHB với Hwdata = 32'hb0000000. Ở phía APB, cầu chuyển đổi bắt đầu thiết lập giao dịch ghi đầu tiên với

PSEL = 4'b0100, PWRITE = 1 và PADDR = 32'h40002fd8. Do APB đang xử lý giao dịch, Hreadyout được kéo xuống thấp để tạm dừng AHB Master.

- Từ T2 đến T3:
 - Giao dịch ghi đầu tiên bước vào pha chuẩn bị của APB. Các tín hiệu PSEL = 4'b0100, PWRITE = 1, PADDR = 32'h40002fd8 và PWDATA = 32'hb0000000 đã được đưa ra ổn định, trong khi PENABLE = 0. Trong khoảng này, Hreadyout vẫn giữ ở mức thấp để duy trì trạng thái chờ ở phía AHB.
- Từ T3 đến T5:
 - PENABLE được đưa lên mức cao, chuyển giao dịch APB sang pha truy cập. Với PREADY = 1 và PSLVERR = 0, Slave xác nhận ghi thành công dữ liệu 32'hb0000000 vào địa chỉ 32'h40002fd8. Sau khi giao dịch hoàn tất, Hreadyout được đưa lên mức cao để AHB Master tiếp tục phát nhịp kế tiếp.
- Từ T5 đến T7:
 - Các nhịp còn lại của burst WRAP4 được xử lý theo cơ chế tương tự. Mỗi nhịp ghi trên APB đều gồm pha chuẩn bị với PENABLE = 0 và pha truy cập với PENABLE = 1. Chuỗi địa chỉ ghi lần lượt là 0x40002fd8, 0x40002fdc, 0x40002fd0, 0x40002fd4; dữ liệu tương ứng là 0xb0000000, 0xb0000001, 0xb0000002, 0xb0000003. Trong suốt quá trình này, Hreadyout được dùng để điều tiết giữa AHB và APB: kéo xuống thấp khi APB đang xử lý và đưa lên cao khi có thể nhận nhịp tiếp theo.
- Từ T7 đến T8:
 - Sau khi nhịp cuối của burst WRAP4 kết thúc, phía AHB đưa Htrans về 2'b00, tương ứng trạng thái IDLE. Ở phía APB, PSEL trở về 4'b0000 và PENABLE = 0, cho biết không còn APB Slave nào đang được truy cập.

Hình 4.5 trình bày kết quả kiểm tra của Scoreboard đối với testcase 2. Trong hình, báo cáo chỉ minh họa hai chuỗi burst tiêu biểu là WRAP4 và INCR4 nhằm tránh lặp lại nội dung không cần thiết ở các chuỗi burst còn lại như INCR8, INCR16, WRAP8 và WRAP16.

TYPE	PKT	BURST	TRANS	DATA (AHB)	DATA (APB)	HADDR	PADDR	SLVERR	HRESP	STATUS
WRITE	[0]	WRAP4	NONSEQ	b0000000	b0000000	40002fd8	40002fd8	0	OKAY	PASS
WRITE	[1]	WRAP4	SEQ	b0000001	b0000001	40002fdc	40002fdc	0	OKAY	PASS
WRITE	[2]	WRAP4	SEQ	b0000002	b0000002	40002fd0	40002fd0	0	OKAY	PASS
WRITE	[3]	WRAP4	SEQ	b0000003	b0000003	40002fd4	40002fd4	0	OKAY	PASS
WRITE	[4]	INCR4	NONSEQ	b0000000	b0000000	400035dc	400035dc	0	OKAY	PASS
WRITE	[5]	INCR4	SEQ	b0000001	b0000001	400035e0	400035e0	0	OKAY	PASS
WRITE	[6]	INCR4	SEQ	b0000002	b0000002	400035e4	400035e4	0	OKAY	PASS
WRITE	[7]	INCR4	SEQ	b0000003	b0000003	400035e8	400035e8	0	OKAY	PASS

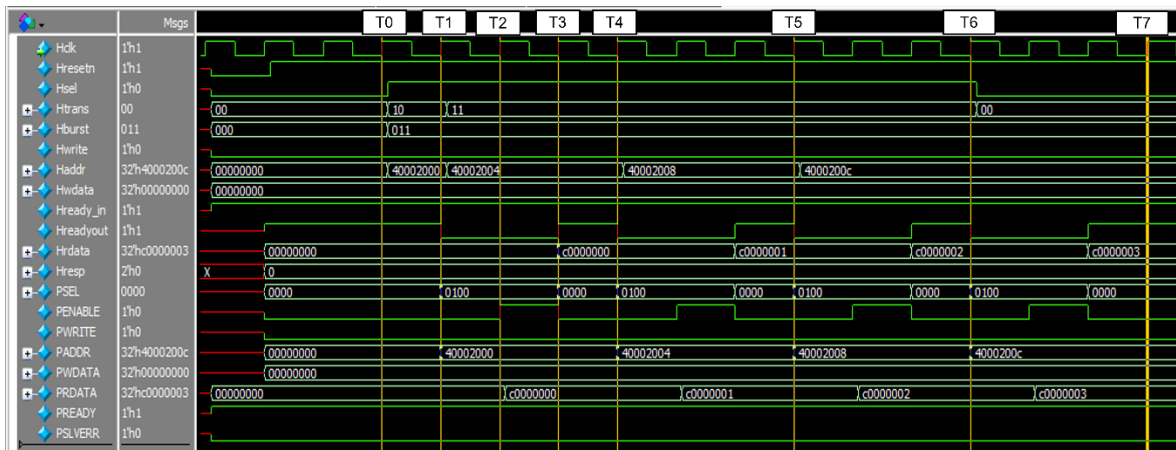
Hình 4.5: Kết quả kiểm tra của Scoreboard ở testcase 2

Kết quả mô phỏng cho thấy các giao dịch trong hai chuỗi burst WRAP4 và INCR4 đều được kiểm tra thành công, với HRESP = OKAY và SLVERR = 0. Địa chỉ HADDR được chuyển đúng sang PADDR, dữ liệu giữa hai phía AHB và APB cũng khớp ở từng nhịp truyền. Trong mỗi chuỗi burst, nhịp đầu tiên sử dụng kiểu NONSEQ, các nhịp còn lại là SEQ, đúng với quy tắc truyền của giao thức AHB. Scoreboard không ghi nhận lỗi nào trong testcase 2, cho thấy cầu chuyển đổi xử lý đúng các giao dịch ghi burst liên tiếp.

4.1.2.3 Testcase 3: Chuỗi giao dịch đọc liên tiếp

Testcase 3 kiểm tra khả năng truyền dữ liệu đọc từ APB Slave về AHB Master khi cầu chuyển đổi xử lý các chuỗi đọc liên tiếp theo nhiều kiểu burst khác nhau.

Hình 4.6 trình bày dạng sóng của kịch bản này, thể hiện hoạt động của các tín hiệu trên hai bus AHB và APB. Trọng tâm kiểm tra là việc dữ liệu PRDATA từ phía APB được chuyển tiếp chính xác về phía AHB thông qua tín hiệu HRDATA.



Hình 4.6: Mô phỏng cầu AHB – APB ở kịch bản chuỗi giao dịch đọc liên tiếp

Kết quả mô phỏng tại hình 4.6 được phân tích theo từng giai đoạn như sau:

- Từ T0 đến T1:
 - AHB Master bắt đầu nhịp đọc đầu tiên của burst INCR4 với Htrans = 2'b10 tương ứng NONSEQ, Hburst = 3'b011 tương ứng INCR4, Hwrite = 0 và Haddr = 32'h40002000. Địa chỉ này thuộc vùng của Slave 2 nên cầu chuyển đổi chọn PSEL = 4'b0100.
- Từ T1 đến T2:
 - Cầu chuyển đổi thiết lập pha chuẩn bị đọc trên APB với PSEL = 4'b0100, PWRITE = 0, PADDR = 32'h40002000 và PENABLE = 0. Đồng thời, Hreadyout được kéo xuống thấp để tạm dừng AHB Master trong khi giao dịch đọc đang được xử lý.
- Từ T2 đến T3:
 - APB chuyển sang pha truy cập khi PENABLE được đưa lên mức cao. Các tín hiệu PSEL, PWRITE và PADDR vẫn giữ ổn định. APB Slave trả dữ liệu đọc đầu tiên qua PRDATA = 32'hc0000000. Do PREADY = 1 và PSLVERR = 0, giao dịch đọc hoàn tất thành công.
- Từ T3 đến T4:
 - Dữ liệu PRDATA = 32'hc0000000 được cầu chuyển đổi đưa về phía AHB thông qua HRDATA = 32'hc0000000. Tín hiệu HRESP = 2'b00 cho biết phản hồi OKAY. Sau đó, PSEL được đưa về 4'b0000,

PENABLE = 0, kết thúc nhịp đọc đầu tiên và chuẩn bị cho nhịp tiếp theo.

- Từ T4 đến T7:
 - Các nhịp đọc còn lại của burst INCR4 được xử lý tương tự. Chuỗi địa chỉ lần lượt là 0x40002000, 0x40002004, 0x40002008, 0x4000200c; dữ liệu đọc tương ứng là 0xc0000000, 0xc0000001, 0xc0000002, 0xc0000003.
 - Ở mỗi nhịp, APB thực hiện pha chuẩn bị với PENABLE = 0, sau đó chuyển sang pha truy cập với PENABLE = 1. Khi PREADY = 1, dữ liệu PRDATA được lấy mẫu và chuyển về AHB qua HRDATA. Trong suốt quá trình này, Hreadyout được dùng để điều tiết giữa hai bus, đảm bảo AHB Master chỉ tiếp tục khi cầu chuyển đổi đã sẵn sàng.

Hình 4.7 thể hiện kết quả Scoreboard của testcase 3, trong đó các chuỗi burst INCR4, INCR8, WRAP4 và WRAP8 được sử dụng làm các trường hợp đại diện cho kịch bản kiểm thử đọc burst.

TYPE	PKT	BURST	TRANS	DATA (AHB)	DATA (APB)	HADDR	PADDR	SLVERR	HRESP	STATUS
READ	[0]	INCR4	NONSEQ	c0000000	c0000000	40002000	40002000	0	OKAY	PASS
READ	[1]	INCR4	SEQ	c0000001	c0000001	40002004	40002004	0	OKAY	PASS
READ	[2]	INCR4	SEQ	c0000002	c0000002	40002008	40002008	0	OKAY	PASS
READ	[3]	INCR4	SEQ	c0000003	c0000003	4000200c	4000200c	0	OKAY	PASS
READ	[4]	INCR8	NONSEQ	c0000004	c0000004	40002100	40002100	0	OKAY	PASS
READ	[5]	INCR8	SEQ	c0000005	c0000005	40002104	40002104	0	OKAY	PASS
READ	[6]	INCR8	SEQ	c0000006	c0000006	40002108	40002108	0	OKAY	PASS
READ	[7]	INCR8	SEQ	c0000007	c0000007	4000210c	4000210c	0	OKAY	PASS
READ	[8]	INCR8	SEQ	c0000008	c0000008	40002110	40002110	0	OKAY	PASS
READ	[9]	INCR8	SEQ	c0000009	c0000009	40002114	40002114	0	OKAY	PASS
READ	[10]	INCR8	SEQ	c000000a	c000000a	40002118	40002118	0	OKAY	PASS
READ	[11]	INCR8	SEQ	c000000b	c000000b	4000211c	4000211c	0	OKAY	PASS
READ	[12]	WRAP4	NONSEQ	c000000c	c000000c	40002208	40002208	0	OKAY	PASS
READ	[13]	WRAP4	SEQ	c000000d	c000000d	4000220c	4000220c	0	OKAY	PASS
READ	[14]	WRAP4	SEQ	c000000e	c000000e	40002200	40002200	0	OKAY	PASS
READ	[15]	WRAP4	SEQ	c000000f	c000000f	40002204	40002204	0	OKAY	PASS
READ	[16]	WRAP8	NONSEQ	c0000010	c0000010	40002310	40002310	0	OKAY	PASS
READ	[17]	WRAP8	SEQ	c0000011	c0000011	40002314	40002314	0	OKAY	PASS
READ	[18]	WRAP8	SEQ	c0000012	c0000012	40002318	40002318	0	OKAY	PASS
READ	[19]	WRAP8	SEQ	c0000013	c0000013	4000231c	4000231c	0	OKAY	PASS
READ	[20]	WRAP8	SEQ	c0000014	c0000014	40002300	40002300	0	OKAY	PASS
READ	[21]	WRAP8	SEQ	c0000015	c0000015	40002304	40002304	0	OKAY	PASS
READ	[22]	WRAP8	SEQ	c0000016	c0000016	40002308	40002308	0	OKAY	PASS
READ	[23]	WRAP8	SEQ	c0000017	c0000017	4000230c	4000230c	0	OKAY	PASS

UVM_INFO testbench/env/bridge_scoreboard.sv(196) @ 1065: uvm_test_top.env.sb [bridge_scoreboard]
 SCOREBOARD TONG KET: PASS = 24 | FAIL = 0

Hình 4.7: Kết quả kiểm tra của Scoreboard ở testcase 3

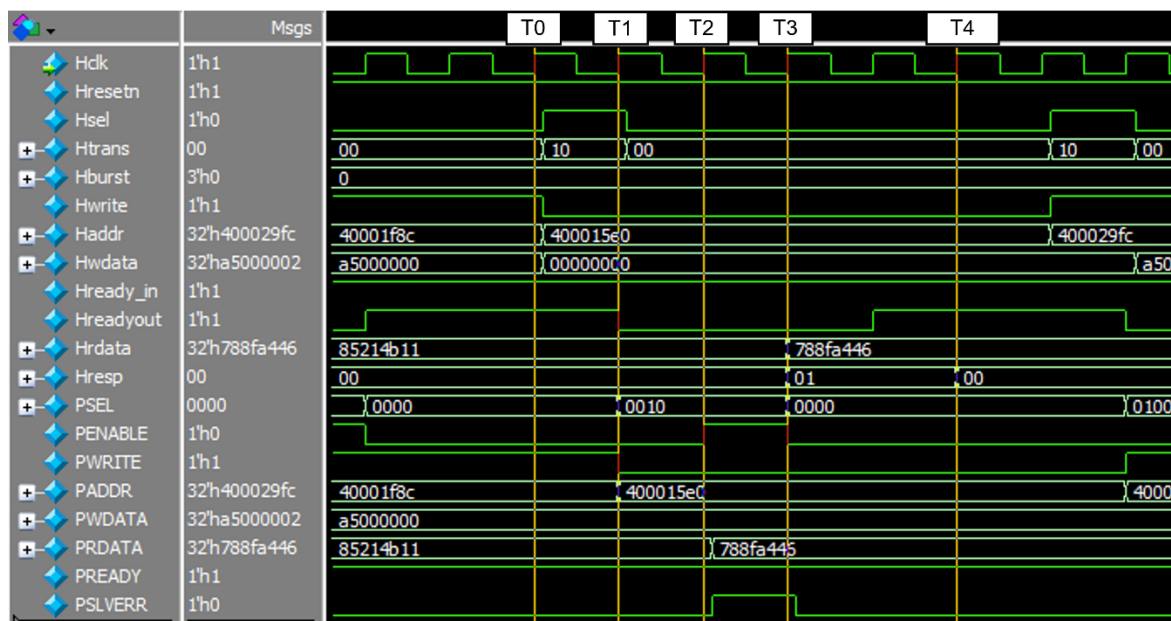
Kết quả cho thấy 24 giao dịch đọc đều được kiểm tra thành công, với HRESP = OKAY và SLVERR = 0. Địa chỉ HADDR được chuyển đúng sang PADDR, đồng thời dữ liệu đọc từ APB khớp với dữ liệu trả về phía AHB ở từng nhịp truyền.

Trong mỗi chuỗi burst, nhịp đầu tiên là NONSEQ và các nhịp tiếp theo là SEQ, đúng với cách truyền của giao thức AHB. Scoreboard ghi nhận PASS = 24 và FAIL = 0, xác nhận cầu chuyển đổi truyền dữ liệu đọc từ APB Slave về AHB Master đúng trong toàn bộ testcase 3.

4.1.2.4 Testcase 4: Phản hồi và truyền tín hiệu báo lỗi từ APB Slave

Testcase 4 kiểm tra khả năng xử lý lỗi khi APB Slave đưa PSLVERR = 1. Mục tiêu của kịch bản là xác nhận cầu chuyển đổi có thể truyền lỗi về phía AHB bằng tín hiệu HRESP = ERROR, đồng thời kết thúc giao dịch lỗi đúng cách và trở lại trạng thái sẵn sàng.

Hình 4.8 kết quả quá trình phản hồi lỗi từ APB sang AHB. Khi PSLVERR được kích hoạt, cầu chuyển đổi giữ HRESP = ERROR trong hai chu kỳ liên tiếp. Ở chu kỳ đầu, Hreadyout = 0 để kéo dài giao dịch; ở chu kỳ sau, Hreadyout = 1 để hoàn tất phản hồi lỗi.



Hình 4.8: Mô phỏng cầu AHB – APB ở kịch bản phản hồi báo lỗi từ APB Slave

Kết quả mô phỏng tại hình 4.8 được phân tích theo từng giai đoạn như sau:

- Từ T0 đến T1:
 - AHB Master phát giao dịch đọc đơn với $Htrans = 2'b10$, $Hburst = 3'b000$, $Hwrite = 0$ và $Haddr = 32'h400015e0$. Địa chỉ này thuộc vùng của Slave 1 nên cầu chuyển đổi chọn $PSEL = 4'b0010$.
- Từ T1 đến T2:
 - Cầu chuyển đổi thiết lập pha chuẩn bị trên APB với $PSEL = 4'b0010$, $PWRITE = 0$, $PADDR = 32'h400015e0$ và $PENABLE = 0$. Đồng thời, $Hreadyout$ được kéo xuống thấp để báo AHB Master chờ trong khi giao dịch đang được xử lý.
- Từ T2 đến T3:
 - APB chuyển sang pha truy cập khi $PENABLE$ được đưa lên mức cao. Các tín hiệu $PSEL$, $PWRITE$ và $PADDR$ vẫn giữ ổn định. APB Slave trả dữ liệu qua $PRDATA = 32'h788fa445$ nhưng đồng thời kích hoạt $PSLVERR = 1$. Vì $PREADY = 1$, giao dịch APB kết thúc, nhưng kết quả được xác định là có lỗi.
- Từ T3 đến T4:
 - Sau khi ghi nhận $PSLVERR$, cầu chuyển đổi phản hồi về AHB bằng $HRESP = 2'b01$, tương ứng trạng thái ERROR. Ở chu kỳ lỗi đầu tiên, $Hreadyout$ được giữ ở mức thấp để kéo dài giao dịch. Sang chu kỳ tiếp theo, $HRESP$ vẫn giữ ERROR, còn $Hreadyout$ được đưa lên cao để AHB Master kết thúc giao dịch lỗi. Do giao dịch có lỗi, dữ liệu đọc không được dùng để kiểm tra kết quả đúng sai.
- Sau T4:
 - Khi phản hồi lỗi hoàn tất, $HRESP$ trở lại $2'b00$ tương ứng OKAY. Cầu chuyển đổi quay về trạng thái rồi và sẵn sàng nhận giao dịch tiếp theo.

Hình 4.9 thể hiện kết quả kiểm tra của Scoreboard trong testcase 4. Kịch bản gồm các giao dịch ghi và đọc đơn xen kẽ, trong đó một số giao dịch được kích hoạt $PSLVERR = 1$ để kiểm tra cơ chế phản hồi lỗi của cầu chuyển đổi.

TYPE	PKT	BURST	TRANS	DATA (AHB)	DATA (APB)	HADDR	PADDR	SLVERR	HRESP	STATUS
WRITE	[0]	SINGLE	NONSEQ	a5000000	a5000000	40003720	40003720	0	OKAY	PASS
READ	[1]	SINGLE	NONSEQ	63183bed	63183bed	400018f0	400018f0	0	OKAY	PASS
WRITE	[2]	SINGLE	NONSEQ	a5000002	a5000002	4000158c	4000158c	0	OKAY	PASS
READ	[3]	SINGLE	NONSEQ	85214b11	85214b11	40002ec0	40002ec0	0	OKAY	PASS
WRITE	[4]	SINGLE	NONSEQ	a5000000	a5000000	40001f8c	40001f8c	0	OKAY	PASS
READ	[5]	SINGLE	NONSEQ	788fa446	788fa446	400015e0	400015e0	1	ERROR	PASS
WRITE	[6]	SINGLE	NONSEQ	a5000002	a5000002	400029fc	400029fc	0	OKAY	PASS
READ	[7]	SINGLE	NONSEQ	7a240f6e	7a240f6e	40002454	40002454	1	ERROR	PASS

UVM_INFO testbench/env/bridge_scoreboard.sv(196) @ 1065: uvm_test_top.env.sb [bridge_scoreboard]
 SCOREBOARD TONG KET: PASS = 8 | FAIL = 0

Hình 4.9: Kết quả kiểm tra của Scoreboard ở testcase 4

Kết quả cho thấy cả 8 giao dịch đều được kiểm tra thành công. Với các giao dịch không có lỗi, SLVERR = 0 và cầu chuyển đổi trả về HRESP = OKAY, dữ liệu giữa phía AHB và APB khớp nhau. Với các giao dịch có lỗi, cầu chuyển đổi truyền lỗi về AHB Master thông qua HRESP = ERROR, cụ thể tại địa chỉ 0x400015e0 và 0x40002454. Scoreboard ghi nhận PASS = 8 và FAIL = 0, xác nhận cơ chế phát hiện và phản hồi lỗi của cầu chuyển đổi hoạt động đúng trong testcase 4.

4.2 ĐÁNH GIÁ CẦU CHUYỂN ĐỔI AHB – APB TRÊN FPGA

Sau khi xác minh chức năng bằng UVM, cầu chuyển đổi AHB – APB được triển khai trên FPGA để đánh giá tài nguyên sử dụng, tần số hoạt động và công suất tiêu thụ. Quá trình này được thực hiện trên thiết bị xc7z045ffg900-2 thuộc dòng Zynq-7000 của Xilinx.

Trong quá trình triển khai, thiết kế được ràng buộc hoạt động ở tần số 100 MHz. Tín hiệu reset bất đồng bộ Hresetn được bỏ qua kiểm tra, trong khi các cổng AHB và APB được thiết lập độ trễ đầu vào và đầu ra bằng 0. Các ràng buộc này được thể hiện ở hình 4.10.

```

1  create_clock -name Hclk_100MHz -period 10.000 [get_ports Hclk]
2  set_false_path -from [get_ports Hresetn]
3  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports Hsel]
4  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {Htrans[*]}]
5  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {Hburst[*]}]
6  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports Hwrite]
7  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {Haddr[*]}]
8  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {Hwdata[*]}]
9  set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports Hready_in]
10 set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {PRDATA[*]}]
11 set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports PREADY]
12 set_input_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports PSLVERR]
13
14 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports Hreadyout]
15 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {Hrdata[*]}]
16 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {Hresp[*]}]
17 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {PSEL[*]}]
18 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports PENABLE]
19 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports PWRITE]
20 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {PADDR[*]}]
21 set_output_delay 0.000 -clock [get_clocks Hclk_100MHz] [get_ports {PWDATA[*]}]

```

Hình 4.10: Cấu hình ràng buộc định thời cho cầu chuyển đổi AHB – APB

4.2.1 Đánh giá tài nguyên sử dụng

Sau khi hoàn tất quá trình triển khai thiết kế, kết quả sử dụng tài nguyên được tổng hợp ở hình 4.11.

Utilization				Post-Synthesis	Post-Implementation
				Graph Table	
Resource	Utilization	Available	Utilization %		
LUT	74	218600	0.03		
FF	210	437200	0.05		
IO	209	362	57.73		
BUFG	1	32	3.13		

Hình 4.11: Kết quả sử dụng tài nguyên sau triển khai thiết kế

Ở hình 4.11, cầu chuyển đổi sử dụng 74 LUT trên tổng 218.600 LUT, tương ứng 0,03%, và 210 Flip-Flop trên tổng 437.200 FF, tương ứng 0,05%. Tỷ lệ sử dụng LUT và FF thấp cho thấy phần logic của thiết kế khá gọn, chủ yếu gồm các thanh ghi, khối điều khiển và máy trạng thái.

Tài nguyên được sử dụng nhiều nhất là IO, với 209 trên tổng 362 cổng, chiếm 57,73%. Nguyên nhân là do thiết kế có nhiều tín hiệu giao tiếp ở cả phía AHB và APB, bao gồm địa chỉ, dữ liệu và các tín hiệu điều khiển. Ngoài ra, thiết kế sử dụng 1 BUFG trên tổng 32 BUFG, tương ứng 3,13%, để phân phối xung nhịp cho hệ thống.

Nhìn chung, kết quả tổng hợp cho thấy cầu chuyển đổi AHB – APB chiếm rất ít tài nguyên logic, phù hợp để tích hợp cùng các khối chức năng khác trong hệ thống lớn hơn.

4.2.2 Đánh giá tần số hoạt động và độ trễ

Sau khi triển khai, Vivado thực hiện phân tích để kiểm tra khả năng hoạt động của thiết kế ở tần số 100 MHz. Kết quả đánh giá bao gồm ba nhóm ràng buộc chính: Setup, Hold và Pulse Width. Kết quả được trình bày ở hình 4.12.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.197 ns	Worst Hold Slack (WHS): 0.052 ns	Worst Pulse Width Slack (WPWS): 4.600 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 517	Total Number of Endpoints: 517	Total Number of Endpoints: 211

All user specified timing constraints are met.

Hình 4.12: Kết quả tóm tắt phân tích định thời của thiết kế

Dựa trên hình 4.12, Vivado cho thấy toàn bộ ràng buộc do người dùng thiết lập đều được thỏa mãn. Thiết kế không ghi nhận vi phạm trên tổng số 517 điểm cuối được kiểm tra.

Đối với Setup, giá trị WNS = +0,197 ns và TNS = 0 cho thấy dữ liệu vẫn ổn định trước cạnh lên xung nhịp, không có đường tín hiệu nào vi phạm ràng buộc tại tần số 100 MHz.

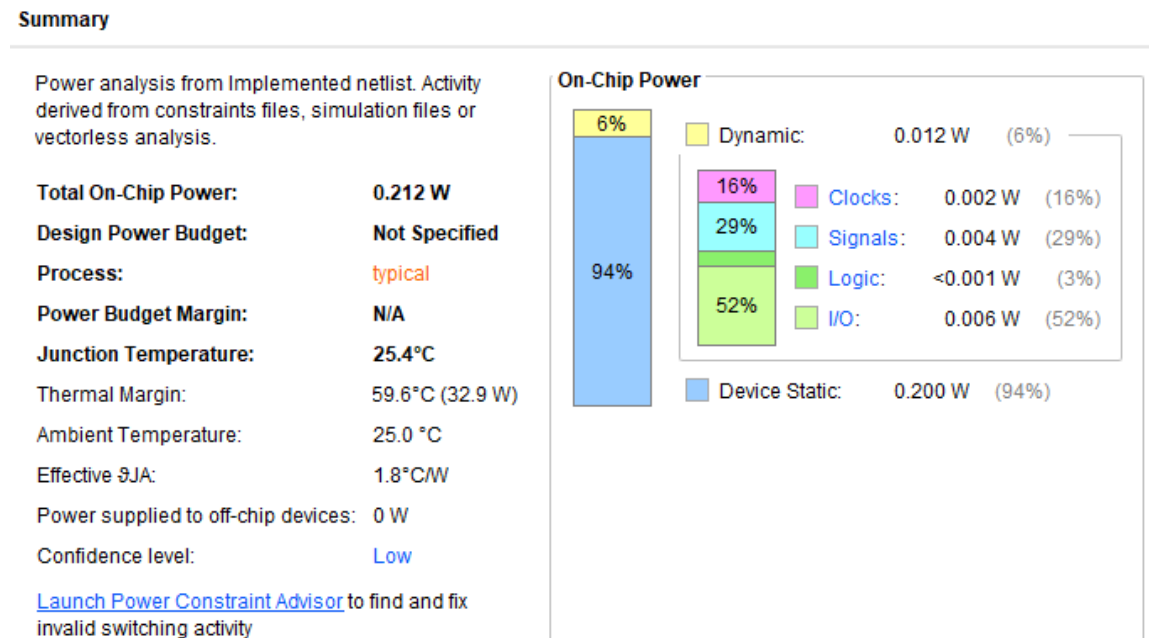
Đối với Hold, giá trị WHS = +0,052 ns và THS = 0 cho thấy dữ liệu vẫn được giữ đủ lâu sau cạnh lên xung nhịp, tránh hiện tượng tín hiệu thay đổi quá sớm gây lỗi lấy mẫu.

Đối với Pulse Width, giá trị WPWS = +4,600 ns trên 211 điểm cuối và không có vi phạm, cho thấy độ rộng xung nhịp đáp ứng yêu cầu hoạt động của các phần tử lưu trữ trong thiết kế.

Nhìn chung, các ràng buộc Setup, Hold và Pulse Width đều đạt với biên an toàn dương, xác nhận cầu chuyển đổi AHB – APB có thể hoạt động ổn định ở tần số 100 MHz.

4.2.3 Đánh giá công suất tiêu thụ

Sau khi hoàn tất kiểm tra các ràng buộc, Vivado tiếp tục ước tính công suất tiêu thụ của thiết kế ở điều kiện nhiệt độ môi trường 25°C. Kết quả được tính dựa trên các thiết lập mặc định của công cụ, không sử dụng dữ liệu hoạt động thực tế từ mô phỏng. Vì vậy, kết quả công suất trong hình 4.13 chỉ mang tính ước lượng, thể hiện mức tiêu thụ dự kiến của thiết kế trong điều kiện phân tích đã chọn.



Hình 4.13: Kết quả báo cáo công suất tiêu thụ của thiết kế

Dựa trên hình 4.13, tổng công suất tiêu thụ của thiết kế là 0,212 W, gồm công suất động và công suất tĩnh.

Công suất động đạt 0,012 W, chiếm khoảng 6% tổng công suất. Trong đó, phần I/O tiêu thụ nhiều nhất với 0,006 W, do thiết kế sử dụng nhiều đường tín hiệu giao tiếp giữa AHB và APB. Các tín hiệu nội bộ tiêu thụ 0,004 W, phần xung nhịp tiêu thụ 0,002 W, còn phần logic tiêu thụ dưới 0,001 W. Điều này cho thấy phần xử lý bên trong của cầu chuyển đổi có mức tiêu thụ rất thấp.

Công suất tĩnh chiếm phần lớn với 0,2 W, tương đương 94% tổng công suất. Đây là công suất nền của thiết bị FPGA, chủ yếu phụ thuộc vào đặc tính phần cứng của chip hơn là hoạt động của riêng cầu chuyển đổi.

Về nhiệt độ, chip ước tính khoảng 25,4°C, thấp hơn nhiều so với giới hạn cho phép. Khoảng dự phòng nhiệt còn 59,6°C, cho thấy thiết kế hoạt động trong vùng an toàn.

4.2.4 Tổng hợp kết quả triển khai

Các kết quả chính sau triển khai được tổng hợp trong Bảng 4.x nhằm thuận tiện cho việc so sánh và nhận xét.

Bảng 4.3: Tổng hợp kết quả triển khai cầu chuyển đổi AHB – APB trên FPGA

Tiêu chí	Kết quả
LUT	74 / 218.600, chiếm 0,03%
Flip-Flop	210 / 437.200, chiếm 0,05%
IO	209 / 362, chiếm 57,73%
BUFG	1 / 32, chiếm 3,13%
WNS	+0,197 ns
WHS	+0,052 ns
WPWS	+4,600 ns
Tổng công suất	0,212 W
Công suất động	0,012 W
Công suất tĩnh	0,2 W
Nhiệt độ chip ước tính	25,4°C

Từ các kết quả trên, có thể thấy cầu chuyển đổi AHB – APB có mức sử dụng tài nguyên logic thấp, đáp ứng được ràng buộc hoạt động ở 100 MHz và có công suất động nhỏ. Mặc dù số lượng IO sử dụng tương đối cao do đặc thù của giao tiếp bus, phần logic bên trong của thiết kế vẫn có quy mô gọn. Các kết quả này cho thấy thiết kế phù hợp để tích hợp vào các hệ thống số lớn hơn trên FPGA.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Đề tài đã hoàn thành các nội dung chính gồm thiết kế cầu chuyển đổi AHB – APB, xây dựng môi trường xác minh UVM và triển khai đánh giá thiết kế trên FPGA.

Về thiết kế, cầu chuyển đổi được xây dựng với hai khối chính là khối giao diện tới AHB và máy trạng thái APB, hỗ trợ kết nối một AHB Master với tối đa bốn APB Slave. Thiết kế đã được kiểm tra với các giao dịch đọc/ghi đơn, các chuỗi giao dịch liên tiếp và trường hợp phản hồi lỗi từ APB Slave.

Về xác minh, kết quả mô phỏng trên môi trường UVM cho thấy cầu chuyển đổi hoạt động đúng trong phạm vi các kịch bản kiểm thử đã xây dựng. Về đánh giá phần cứng, thiết kế được tổng hợp thành công trên Vivado, qua đó đánh giá được tài nguyên sử dụng, khả năng hoạt động ở tần số 100 MHz và công suất tiêu thụ.

Tuy nhiên, đề tài vẫn còn một số hạn chế như chỉ hỗ trợ một AHB Master, số lượng APB Slave giới hạn ở bốn thiết bị, chưa xét trường hợp AHB và APB dùng hai miền xung nhịp độc lập, chưa đánh giá độ bao phủ kiểm thử và chưa chạy thử trực tiếp trên phần cứng FPGA.

5.2 HƯỚNG PHÁT TRIỂN

Từ những hạn chế hiện tại, đề tài có thể được phát triển theo hướng hỗ trợ nhiều AHB Master, mở rộng số lượng APB Slave và bổ sung khả năng cấu hình linh hoạt cho từng vùng địa chỉ. Ngoài ra, môi trường UVM cần được bổ sung phần đánh giá độ bao phủ chức năng để kiểm tra thiết kế đầy đủ và có hệ thống hơn.

Bên cạnh mô phỏng, thiết kế cũng nên được triển khai và chạy thử trực tiếp trên bo mạch FPGA để đánh giá hoạt động trong điều kiện phần cứng thực tế.

Cầu chuyển đổi AHB – APB và môi trường xác minh UVM trong đề tài có thể được dùng làm nền tảng cho các hướng nghiên cứu tiếp theo. Các thành phần trong môi trường UVM có thể được mở rộng hoặc điều chỉnh độc lập, giúp thuận tiện hơn khi áp dụng cho các thiết kế tương tự sau này.

TÀI LIỆU THAM KHẢO

- [1] ARM Ltd., AMBA 3 AHB-Lite Protocol Specification (ARM IHI 0033A). Cambridge: ARM Limited, 2010. Available at: <https://developer.arm.com/documentation/ih0033/latest>.
- [2] ARM Ltd., AMBA APB Protocol Specification (ARM IHI 0024E). Cambridge: ARM Limited, 2023. Available at: <https://developer.arm.com/documentation/ih0024/latest>.
- [3] R. M. Vani and M. Roopa, "Design of AMBA Based AHB2APB Bridge," International Journal of Computer Science and Network Security (IJCSNS), 2010. Available at: http://paper.ijcsns.org/07_book/201011/20101104.pdf.
- [4] Accellera Systems Initiative, Universal Verification Methodology 1.2 User's Guide. Accellera Systems Initiative, 2017. Available at: https://accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf.
- [5] Colorlesscube, "UVM Guide for Beginners – Chapter 2: Defining the Verification Environment," Colorlesscube, n.d. Available at: <https://colorlesscube.com/uvm-guide-for-beginners/chapter-2-defining-the-verification-environment/1000/>.
- [6] Verification for All, "UVM Phases," Verification for All, n.d. Available at: <https://verificationforall.wordpress.com/uvm-phases/>.
- [7] B. Sajidha Thabassum, S. P. Kishan, and D. Nagesh, "Design and Implementation of AHB to APB Bridge Using Verilog," International Journal of Creative Research Thoughts (IJCRT), vol. 13, no. 1, pp. c548–c553, 2025. Available at: <https://ijcrt.org/papers/IJCRT2501292.pdf>.
- [8] G. Elamathi, "Design and Verification of AHB to APB Bridge Protocol Using UVM," International Journal of Scientific Research & Engineering Trends, vol. 10, no. 3, pp. 707–712, 2024. Available at: https://ijsret.com/wp-content/uploads/2024/05/IJSRET_V10_issue3_213.pdf.

PHỤ LỤC

Để tiện cho việc truy cập và kiểm tra mã nguồn của đề tài, mã nguồn có thể được truy cập thông qua đường dẫn dưới đây:

https://github.com/lngle23/AHB_to_APB_Bridge_UVM